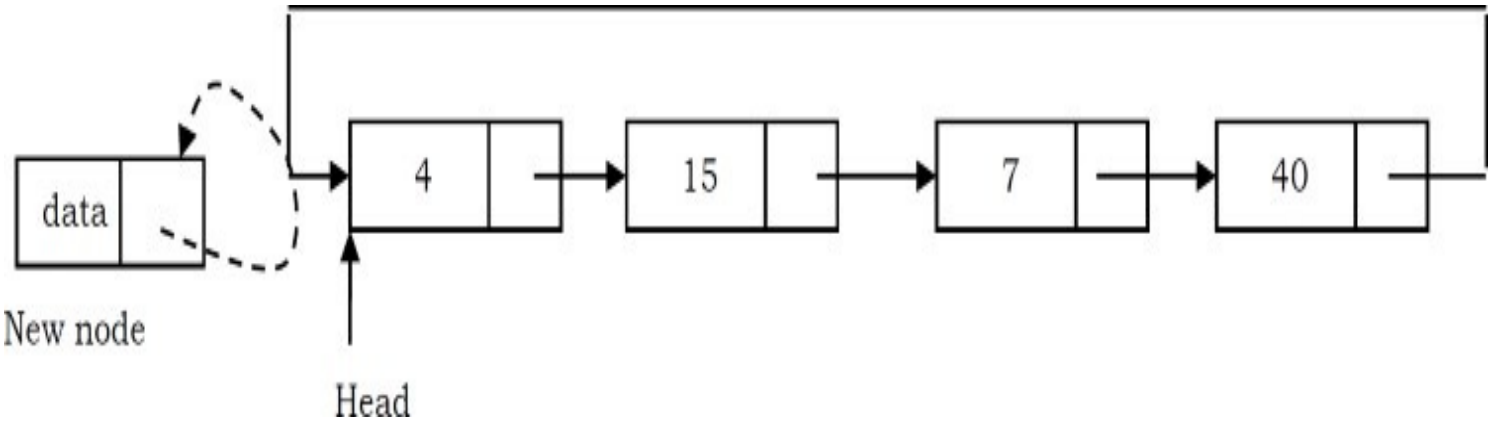
```
void InsertAtEndInCLL (struct CLLNode **head, int data) {
    struct CLLNode *current = *head;
    struct CLLNode *newNode = (struct CLLNode *) (malloc(sizeof(struct CLLNode)));
    if(!newNode) {
        printf("Memory Error");
        return;
    }
    newNode→data = data;
    while (current→next != *head)
        current = current→next;

    newNode→next = newNode;

    if(*head ==NULL)
        *head = newNode;
    else {
        newNode→next = *head;
        current→next = newNode;
    }
}
```

Time Complexity: O(*n*), for scanning the complete list of size *n*.
Space Complexity: O(1), for temporary variable.

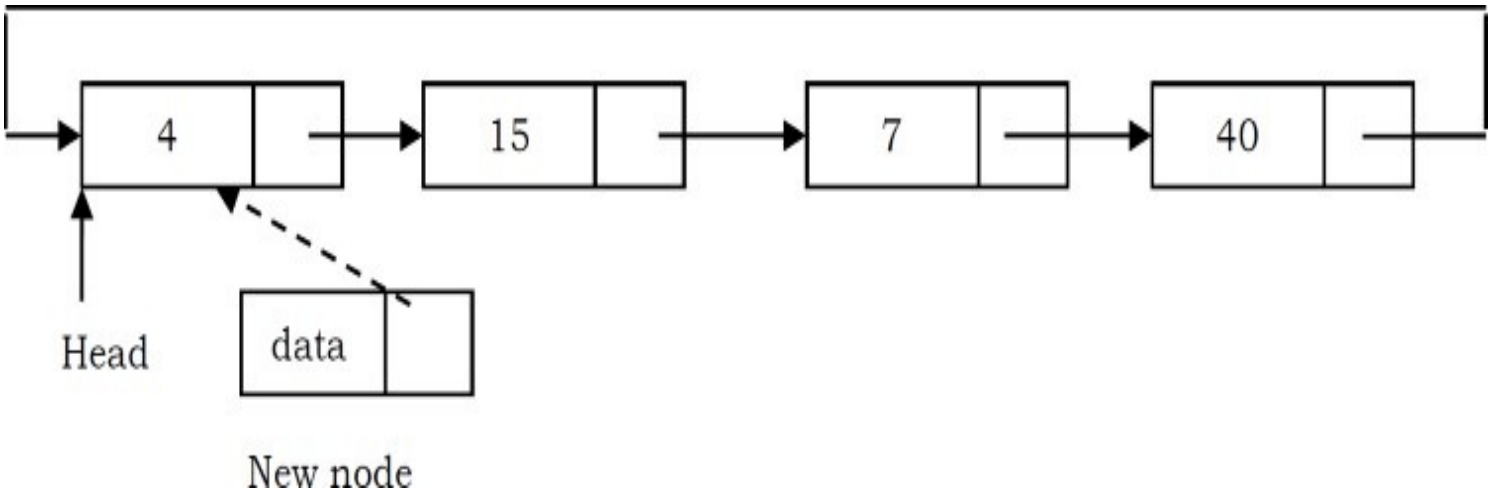
Inserting a Node at the Front of a Circular Linked List

The only difference between inserting a node at the beginning and at the end is that, after inserting the new node, we just need to update the pointer. The steps for doing this are given below:

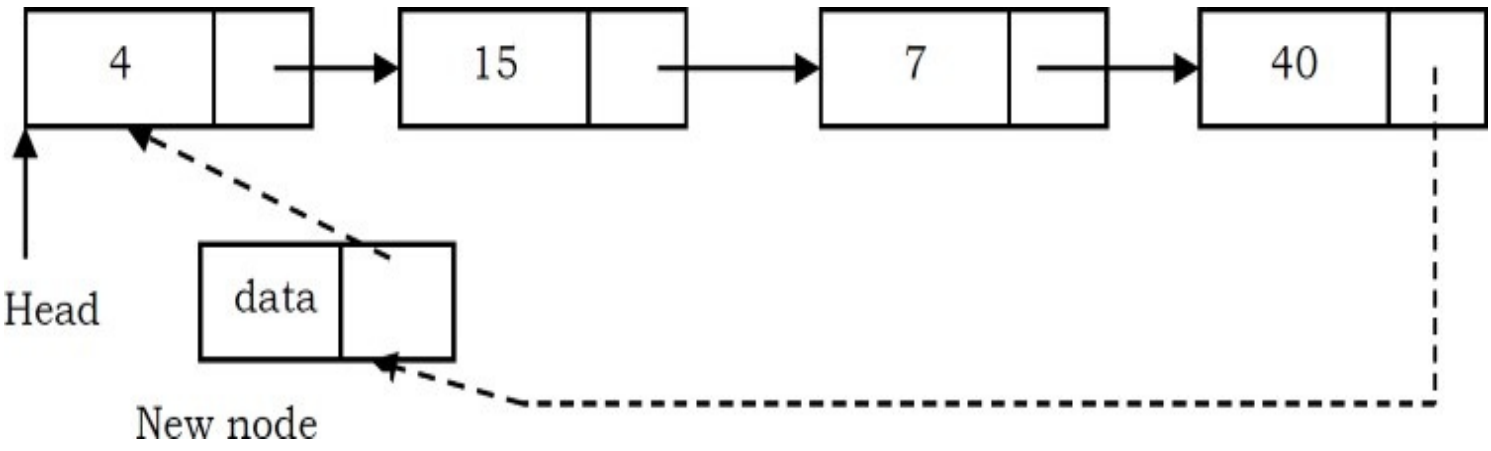
- Create a new node and initially keep its next pointer pointing to itself.



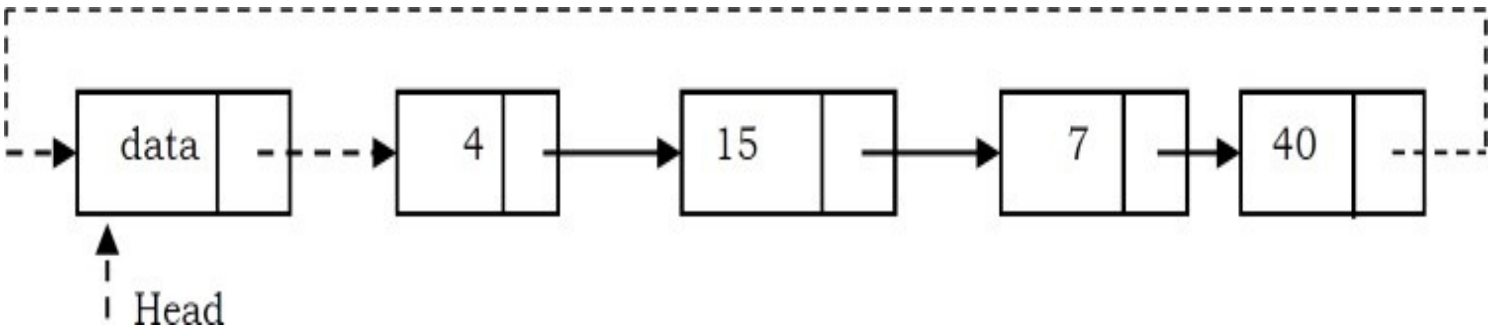
- Update the next pointer of the new node with the head node and also traverse the list until the tail. That means in a circular list we should stop at the node which is its previous node in the list.



- Update the previous head node in the list to point to the new node.



- Make the new node as the head.




```
void InsertAtBeginInCLL (struct CLLNode **head, int data) {
    struct CLLNode *current = *head;
    struct CLLNode * newNode = (struct CLLNode *) (malloc(sizeof(struct CLLNode)));
    if(!newNode) {
        printf("Memory Error");
        return;
    }
    newNode→data = data;
    while (current→next != *head)
        current = current→next;
    newNode→next = newNode;
    if(*head ==NULL)
        *head = newNode;
    else {
        newNode→next = *head;
        current→next = newNode;
        *head = newNode;
    }
    return;
}
```

Time Complexity: O(*n*), for scanning the complete list of size *n*.
Space Complexity: O(1), for temporary variable.

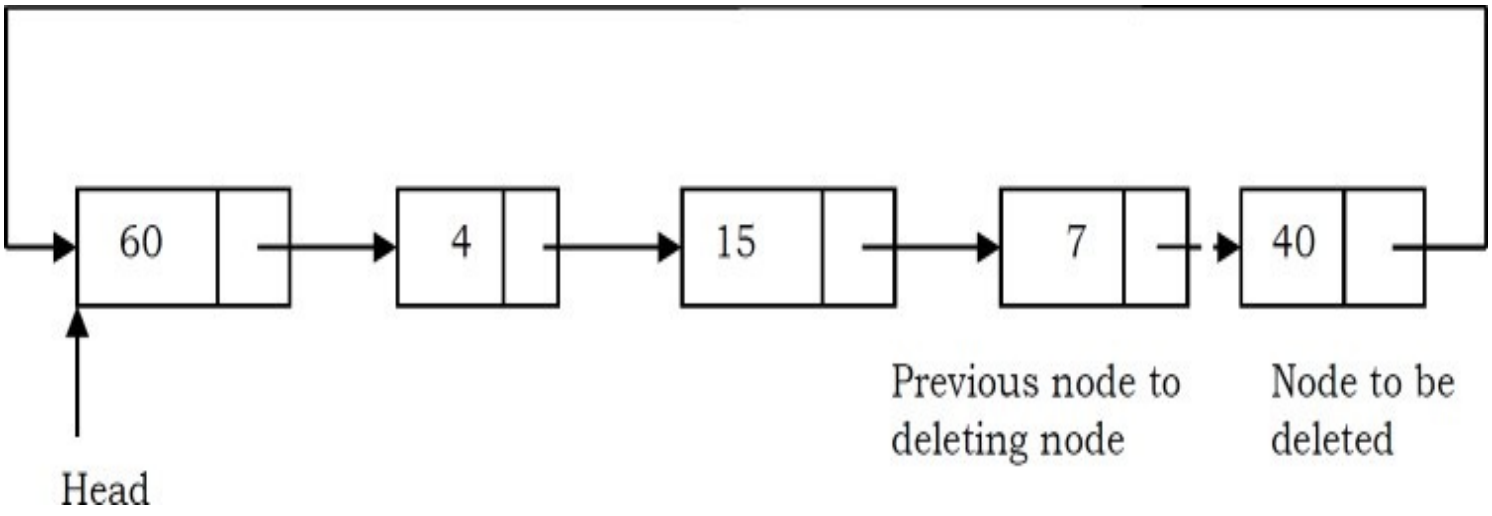
Deleting the Last Node in a Circular Linked List

The list has to be traversed to reach the last but one node. This has to be named as the tail node, and its next field has to point to the first node. Consider the following list.

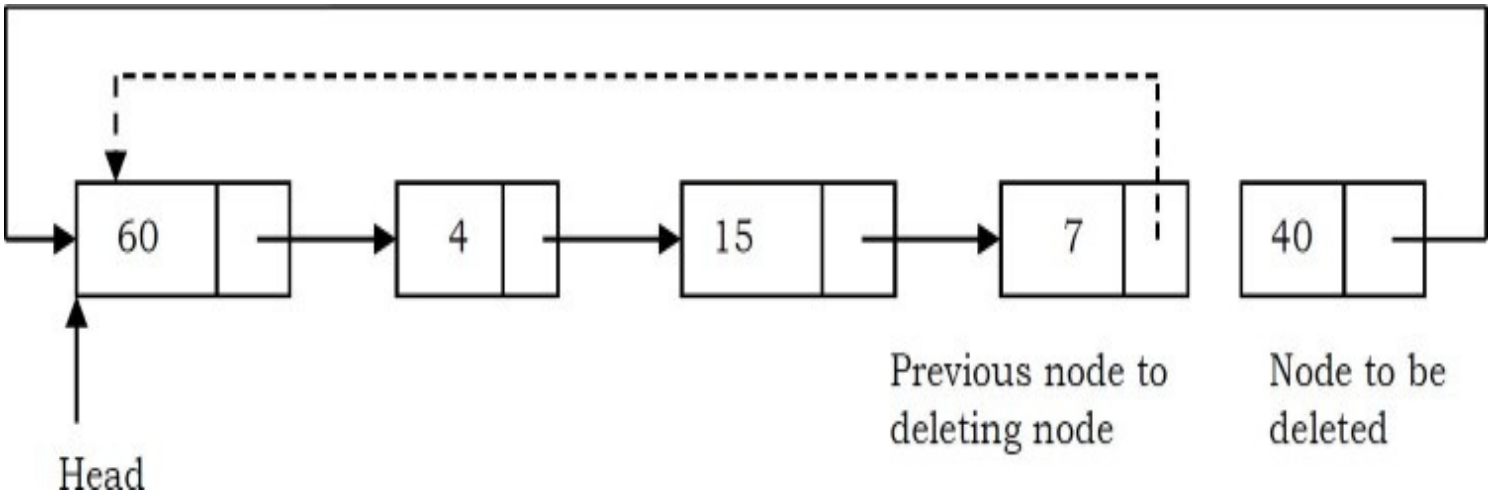
To delete the last node 40, the list has to be traversed till you reach 7. The next field of 7 has to

be changed to point to 60, and this node must be renamed *pTail*.

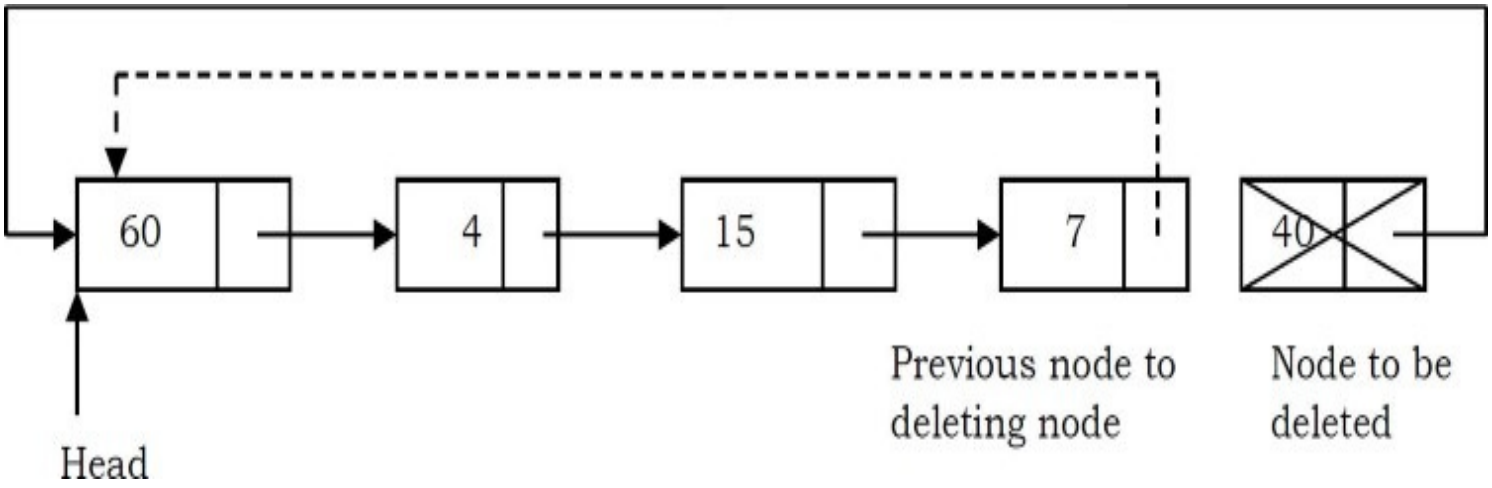
- Traverse the list and find the tail node and its previous node.



- Update the next pointer of tail node's previous node to point to head.



- Dispose of the tail node.



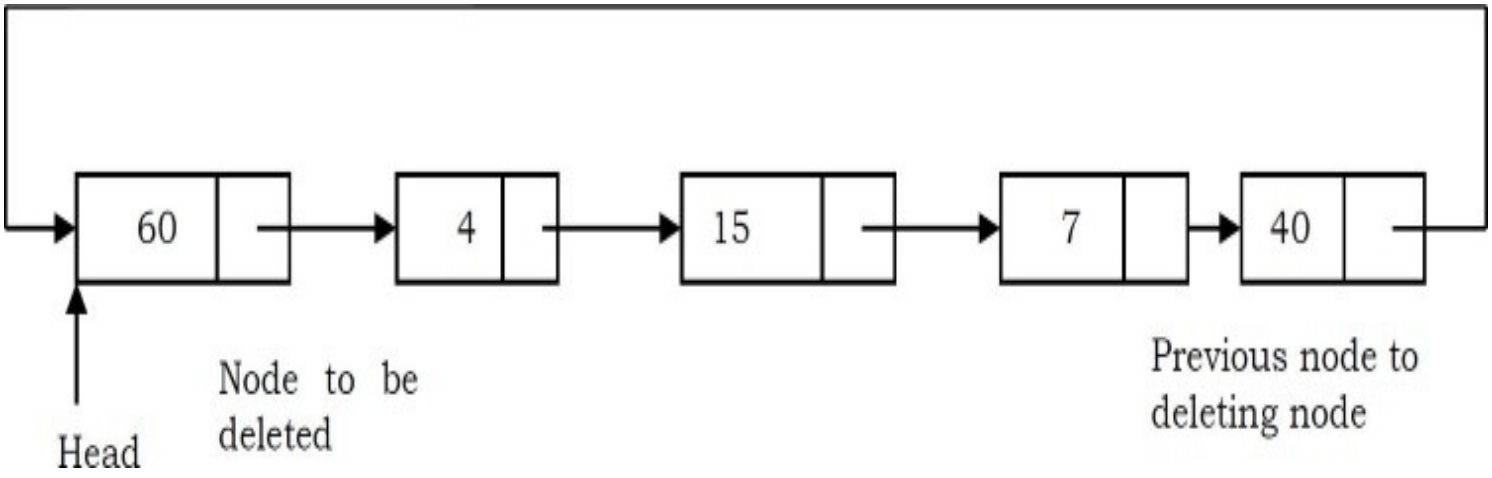
```
void DeleteLastNodeFromCLL (struct CLLNode **head) {
    struct CLLNode *temp = *head, *current = *head;
    if(*head == NULL) {
        printf( "List Empty");  return;
    }
    while (current→next != *head) {
        temp = current;
        current = current→next;
    }
    temp→next = current→next;
    free(current);
    return;
}
```

Time Complexity: O(*n*), for scanning the complete list of size *n*. Space Complexity: O(1), for a temporary variable.

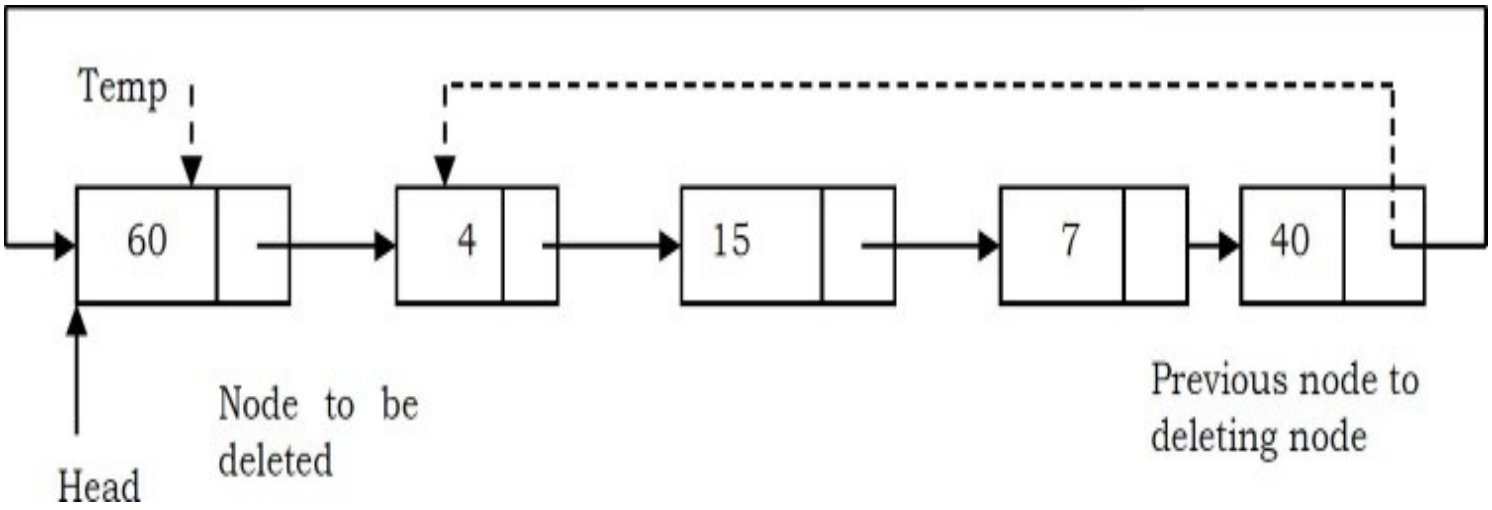
Deleting the First Node in a Circular List

The first node can be deleted by simply replacing the next field of the tail node with the next field of the first node.

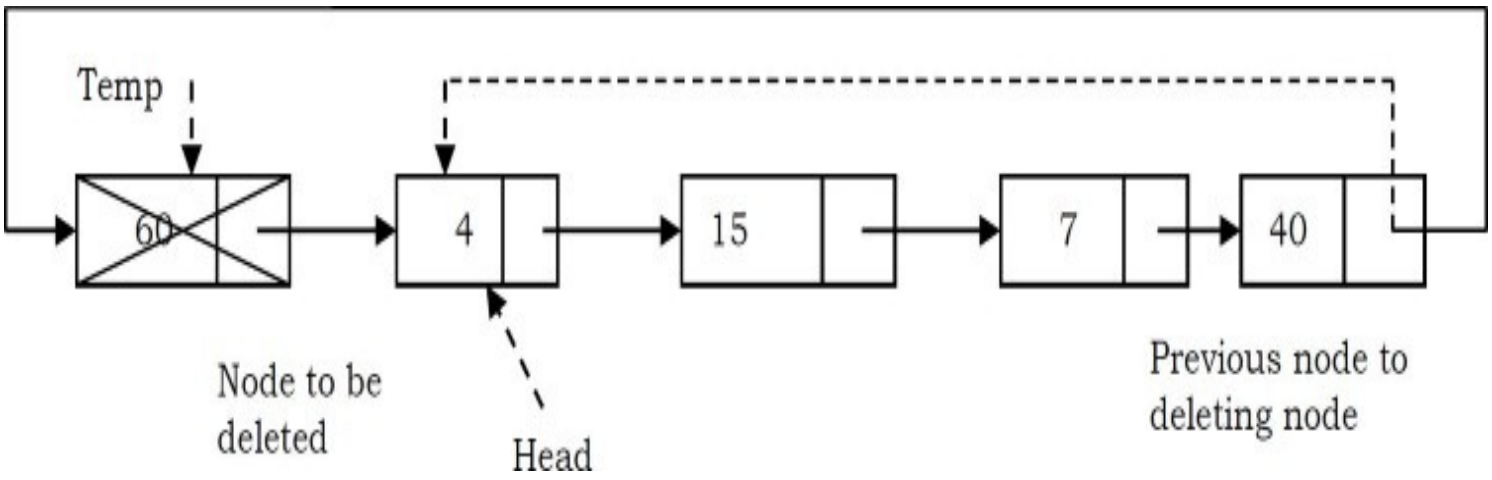
- Find the tail node of the linked list by traversing the list. Tail node is the previous node to the head node which we want to delete.



- Create a temporary node which will point to the head. Also, update the tail nodes next pointer to point to next node of head (as shown below).



- Now, move the head pointer to next node. Create a temporary node which will point to head. Also, update the tail nodes next pointer to point to next node of head (as shown below).



```
void DeleteFrontNodeFromCLL (struct CLLNode **head) {
    struct CLLNode *temp = *head;
    struct CLLNode *current = *head;

    if(*head == NULL) {
       printf("List Empty");
       return;
    }
    while (current→next != *head)
       current = current→next;

    current→ next = *head→next;
    *head = *head→next;
    free(temp);
    return;
}
```

Time Complexity: O(*n*), for scanning the complete list of size *n*.
Space Complexity: O(1), for a temporary variable.

Applications of Circular List

Circular linked lists are used in managing the computing resources of a computer. We can use circular lists for implementing stacks and queues.

3.9 A Memory-efficient Doubly Linked List

In conventional implementation, we need to keep a forward pointer to the next item on the list and a backward pointer to the previous item. That means elements in doubly linked list implementations consist of data, a pointer to the next node and a pointer to the previous node in the list as shown below.

Conventional Node Definition

```
typedef struct ListNode {
   int data;
   struct ListNode * prev;
   struct ListNode * next;
};
```

Recently a journal (Sinha) presented an alternative implementation of the doubly linked list ADT, with insertion, traversal and deletion operations. This implementation is based on pointer difference. Each node uses only one pointer field to traverse the list back and forth.

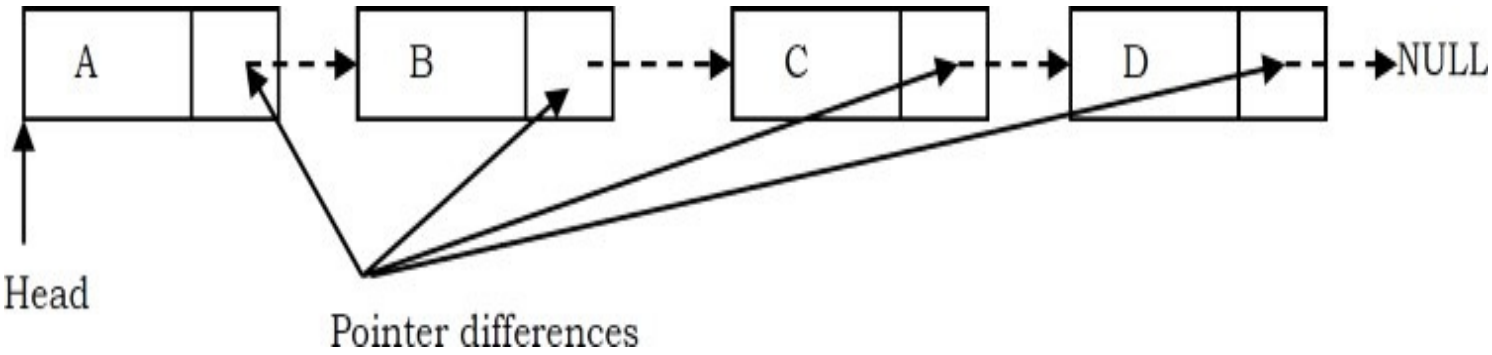
New Node Definition

```
typedef struct ListNode {
 int data;
 struct ListNode * ptrdiff;
};
```

The *ptrdiff* pointer field contains the difference between the pointer to the next node and the pointer to the previous node. The pointer difference is calculated by using exclusive-or (⊕) operation.

ptrdiff = pointer to previous node ⊕ pointer to next node.

The *ptrdiff* of the start node (head node) is the ⊕ of NULL and *next* node (*n*ext node to head). Similarly, the *ptrdiff* of end node is the ⊕ of *previous* node (previous to end node) and NULL. As an example, consider the following linked list.



In the example above,

- The next pointer of A is: NULL ⊕ B
- The next pointer of B is: A ⊕ C
- The next pointer of C is: B ⊕ D
- The next pointer of D is: C ⊕ NULL

Why does it work?

To find the answer to this question let us consider the properties of ⊕:

X ⊕ X=0
X ⊕ 0 = X
X ⊕ Y = Y ⊕ X (symmetric)
(X ⊕ Y) ⊕ Z = X ⊕ (Y ⊕ Z) (transitive)

For the example above, let us assume that we are at C node and want to move to B. We know that C's *ptrdiff* is defined as B ⊕ D. If we want to move to B, performing ⊕ on C's *ptrdiff* with D would give B. This is due to the fact that

(B ⊕ D) ⊕ D = B(since, D ⊕ D= 0)

Similarly, if we want to move to D, then we have to apply ⊕ to C's *ptrdiff* with B to give D.

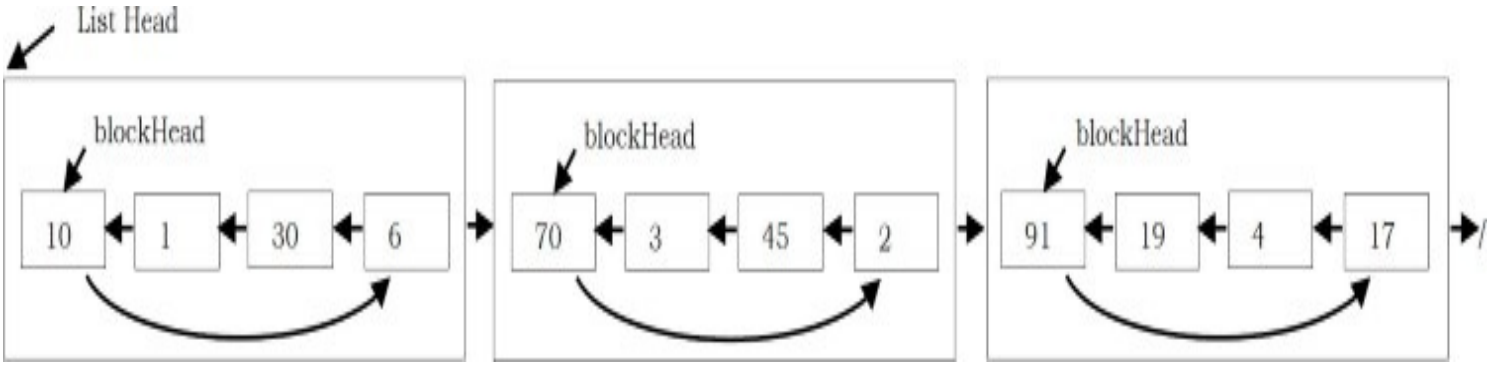
(B ⊕ D) ⊕ B = D (since, B © B=0)

From the above discussion we can see that just by using a single pointer, we can move back and forth. A memory-efficient implementation of a doubly linked list is possible with minimal compromising of timing efficiency.

3.10 Unrolled Linked Lists

One of the biggest advantages of linked lists over arrays is that inserting an element at any location takes only O(1) time. However, it takes O(*n*) to search for an element in a linked list. There is a simple variation of the singly linked list called *unrolled linked lists.*

An unrolled linked list stores multiple elements in each node (let us call it a block for our convenience). In each block, a circular linked list is used to connect all nodes.



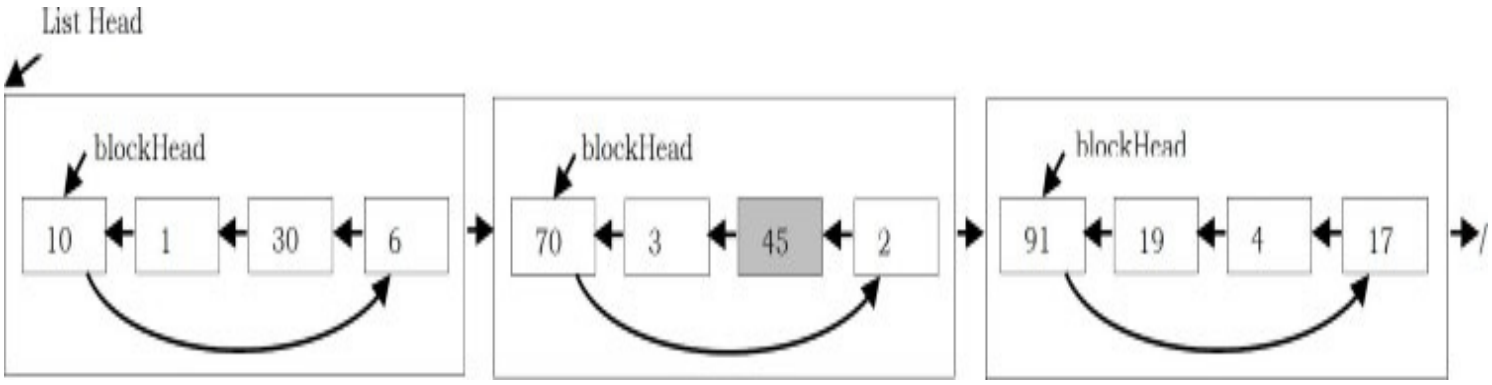
Assume that there will be no more than n elements in the unrolled linked list at any time. To simplify this problem, all blocks, except the last one, should contain exactly $\lceil \sqrt{n} \rceil$ elements. Thus,

there will be no more than $\lceil\sqrt{n}\rceil$ blocks at any time.

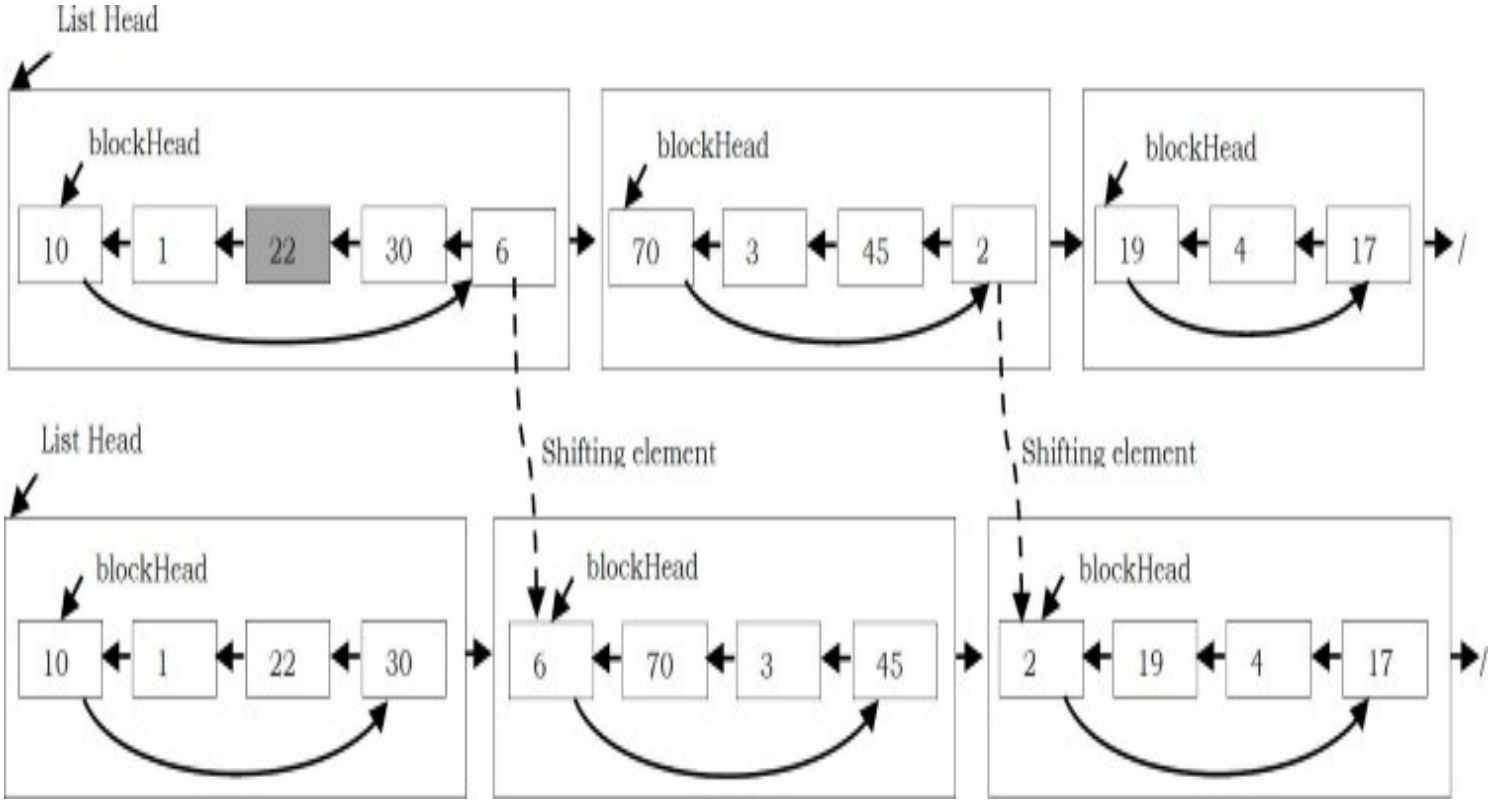
Searching for an element in Unrolled Linked Lists

In unrolled linked lists, we can find the k^{th} element in O($\sqrt{n}$):

1. Traverse the *list of blocks* to the one that contains the k^{th} node, i.e., the $\left\lceil\frac{k}{\lceil\sqrt{n}\rceil}\right\rceil^{\text{th}}$ block. It takes O($\sqrt{n}$) since we may find it by going through no more than $\sqrt{n}$ blocks.
2. Find the (k mod $\lceil\sqrt{n}\rceil$)$^{\text{th}}$ node in the circular linked list of this block. It also takes O($\sqrt{n}$) since there are no more than $\lceil\sqrt{n}\rceil$ nodes in a single block.



Inserting an element in Unrolled Linked Lists

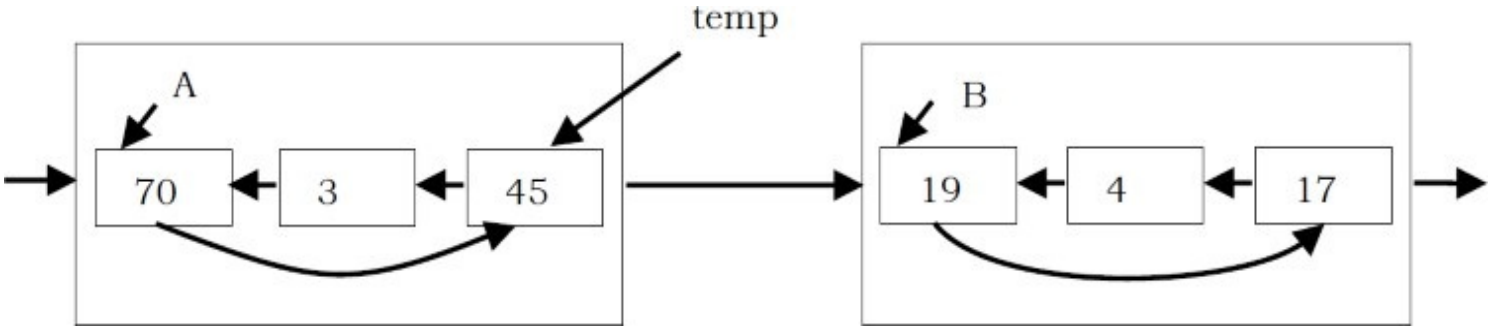


When inserting a node, we have to re-arrange the nodes in the unrolled linked list to maintain the properties previously mentioned, that each block contains $\lceil\sqrt{n}\rceil$ nodes. Suppose that we insert a node *x* after the i^{th} node, and *x* should be placed in the j^{th} block. Nodes in the j^{th} block and in the blocks after the j^{th} block have to be shifted toward the tail of the list so that each of them still have $\lceil\sqrt{n}\rceil$ nodes. In addition, a new block needs to be added to the tail if the last block of the list is out of space, i.e., it has more than $\lceil\sqrt{n}\rceil$ nodes.

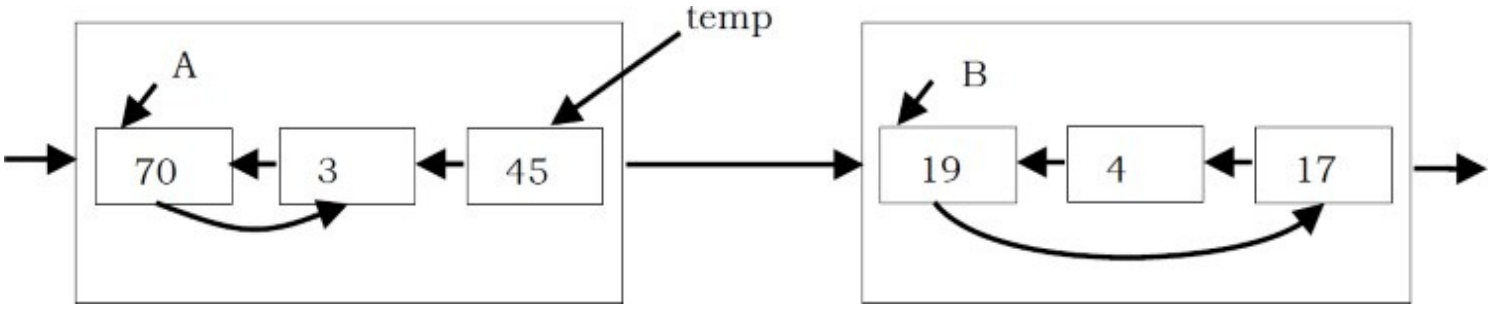
Performing Shift Operation

Note that each *shift* operation, which includes removing a node from the tail of the circular linked list in a block and inserting a node to the head of the circular linked list in the block after, takes only O(1). The total time complexity of an insertion operation for unrolled linked lists is therefore $O(\sqrt{n})$; there are at most $O(\sqrt{n})$ blocks and therefore at most $O(\sqrt{n})$ shift operations.

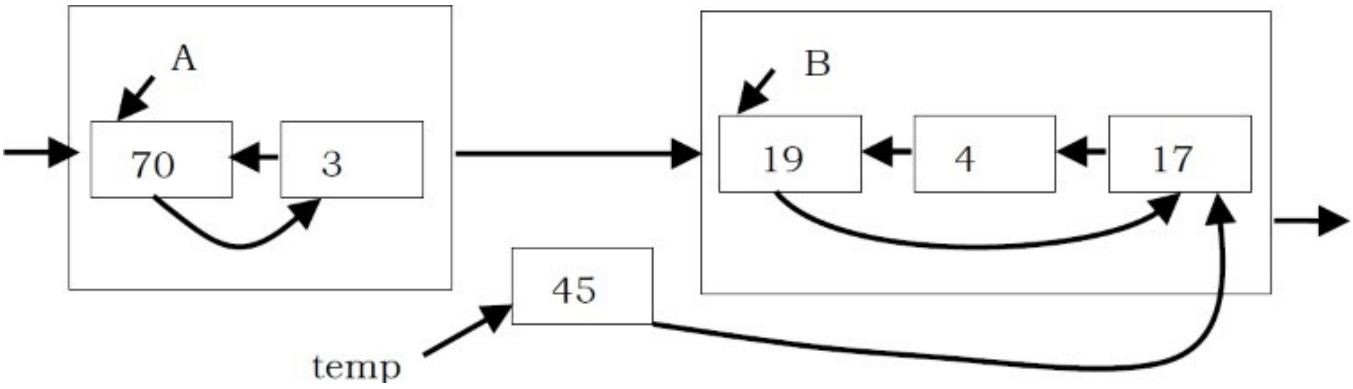
1. A temporary pointer is needed to store the tail of *A*.



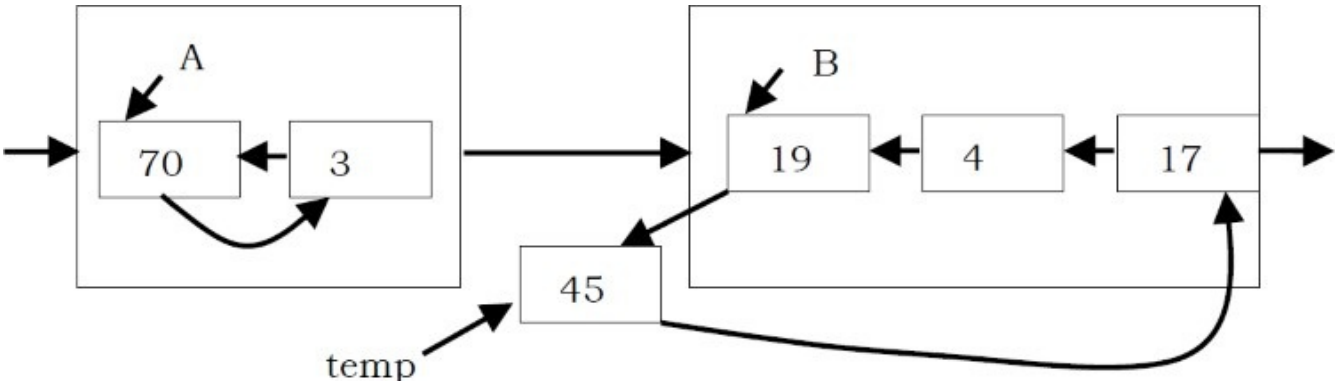
2. In block *A*, move the next pointer of the head node to point to the second-to-last node, so that the tail node of *A* can be removed.



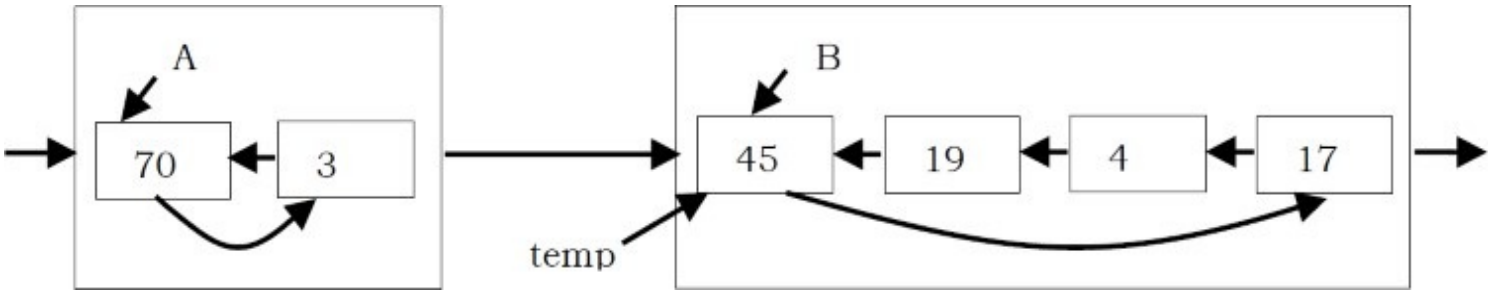
3. Let the next pointer of the node, which will be shifted (the tail node of *A*), point to the tail node of *B*.



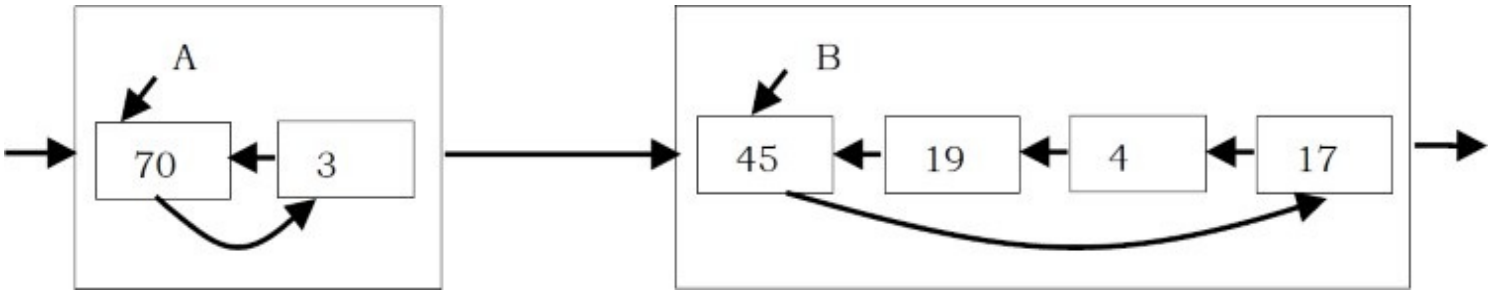
4. Let the next pointer of the head node of *B* point to the node *temp* points to.



5. Finally, set the head pointer of *B* to point to the node *temp* points to. Now the node *temp* points to becomes the new head node of *B*.



6. *temp* pointer can be thrown away. We have completed the shift operation to move the original tail node of *A* to become the new head node of *B*.



Performance

With unrolled linked lists, there are a couple of advantages, one in speed and one in space. First, if the number of elements in each block is appropriately sized (e.g., at most the size of one cache line), we get noticeably better cache performance from the improved memory locality. Second, since we have O(*n*/*m*) links, where *n* is the number of elements in the unrolled linked list and *m* is the number of elements we can store in any block, we can also save an appreciable amount of space, which is particularly noticeable if each element is small.

Comparing Linked Lists and Unrolled Linked Lists

To compare the overhead for an unrolled list, elements in doubly linked list implementations consist of data, a pointer to the next node, and a pointer to the previous node in the list, as shown below.

```
struct ListNode {
    int data;
    struct ListNode *prev;
    struct ListNode *next;
};
```

Assuming we have 4 byte pointers, each node is going to take 8 bytes. But the allocation overhead for the node could be anywhere between 8 and 16 bytes. Let's go with the best case and assume it will be 8 bytes. So, if we want to store IK items in this list, we are going to have 16KB of overhead.

Now, let's think about an unrolled linked list node (let us call it *LinkedBlock*). It will look something like this:

```
struct LinkedBlock{
    struct LinkedBlock *next;
    struct ListNode *head;
    int nodeCount;
};
```

Therefore, allocating a single node (12 bytes + 8 bytes of overhead) with an array of 100 elements (400 bytes + 8 bytes of overhead) will now cost 428 bytes, or 4.28 bytes per element. Thinking about our IK items from above, it would take about 4.2KB of overhead, which is close to 4x better than our original list. Even if the list becomes severely fragmented and the item arrays are only 1/2 full on average, this is still an improvement. Also, note that we can tune the array size to whatever gets us the best overhead for our application.

Implementation

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <math.h>
#include <time.h>

int blockSize; //max number of nodes in a block

struct ListNode{
    struct ListNode* next;
    int value;
};
```

```
struct LinkedBlock{
    struct LinkedBlock *next;
    struct ListNode *head;
    int nodeCount;
};
struct LinkedBlock* blockHead;

//create an empty block
struct LinkedBlock* newLinkedBlock(){
    struct LinkedBlock* block=(struct LinkedBlock*)malloc(sizeof(struct LinkedBlock));
    block→next=NULL;
    block→head=NULL;
    block→nodeCount=0;
    return block;
}
//create a node
struct ListNode* newListNode(int value){
    struct ListNode* temp=(struct ListNode*)malloc(sizeof(struct ListNode));
    temp→next=NULL;
    temp→value=value;
    return temp;
}
void searchElement(int k,struct LinkedBlock **fLinkedBlock,struct ListNode **fListNode){
    //find the block
    int j=(k+blockSize-1)/blockSize; //k-th node is in the j-th block
    struct LinkedBlock* p=blockHead;
    while(--j){
        p=p→next;
    }
    *fLinkedBlock=p;
    //find the node
    struct ListNode* q=p→head;
    k=k%blockSize;
    if(k==0) k=blockSize;
    k=p→nodeCount+1-k;
    while(k--){
        q=q→next;
    }
    *fListNode=q;
}
//start shift operation from block *p
void shift(struct LinkedBlock *A){
    struct LinkedBlock *B;
    struct ListNode* temp;
    while(A→nodeCount > blockSize){ //if this block still have to shift
        if(A→next==NULL){ //reach the end. A little different
            A→next=newLinkedBlock();
            B=A→next;
            temp=A→head→next;
            A→head→next=A→head→next→next;
            B→head=temp;
            temp→next=temp;
            A→nodeCount--;
            B→nodeCount++;
        }else{
            B=A→next;
            temp=A→head→next;
            A→head→next=A→head→next→next;
            temp→next=B→head→next;
            B→head→next=temp;
            B→head=temp
            A→nodeCount--;
```

```
            B→nodeCount++;
        }
        A=B;
    }
}
void addElement(int k,int x){
    struct ListNode *p,*q;
    struct LinkedBlock *r;

    if(!blockHead){ //initial, first node and block
        blockHead=newLinkedBlock();
        blockHead→head=newListNode(x);
        blockHead→head→next=blockHead→head;
        blockHead→nodeCount++;
    }else{
        if(k==0){ //special case for k=0.
        p=blockHead→head;
        q=p→next;
        p→next=newListNode(x);
        p→next→next=q;
        blockHead→head=p→next;
        blockHead→nodeCount++;
        shift(blockHead);
        }else{
            searchElement(k,&r,&p);
            q=p;
            while(q→next!=p) q=q→next;
            q→next=newListNode(x);
            q→next→next=p;
            r→nodeCount++;
            shift(r);
        }
    }
}
int searchElement(int k){
    struct ListNode *p;
    struct LinkedBlock *q;
    searchElement(k,&q,&p);
    return p→value;
}
int testUnRolledLinkedList(){
    int tt=clock();
    int m,i,k,x;
    char cmd[10];
    scanf("%d",&m);
    blockSize=(int)(sqrt(m-0.001))+1;

    for( i=0; i<m; i++ ){
        scanf("%s",cmd);
        if(strcmp(cmd,"add")==0){
            scanf("%d %d",&k,&x);
            addElement(k,x);
        }else if(strcmp(cmd,"search")==0){
            scanf("%d",&k);
            printf("%d\n",searchElement(k));
        }else{
             fprintf(stderr,"Wrong Input\n");
        }
    }
    return 0;
}
```

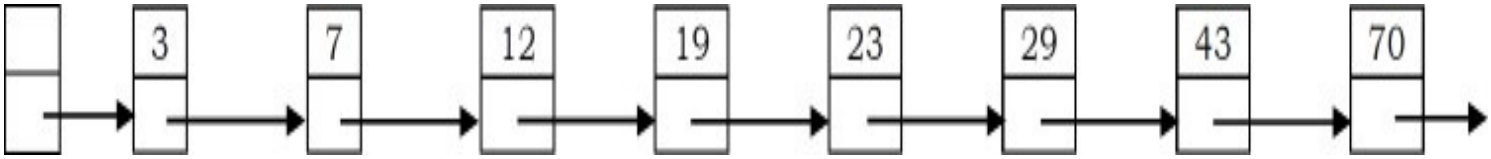
3.11 Skip Lists

Binary trees can be used for representing abstract data types such as dictionaries and ordered lists. They work well when the elements are inserted in a random order. Some sequences of operations, such as inserting the elements in order, produce degenerate data structures that give very poor performance. If it were possible to randomly permute the list of items to be inserted, trees would work well with high probability for any input sequence. In most cases queries must be answered on-line, so randomly permuting the input is impractical. Balanced tree algorithms re-arrange the tree as operations are performed to maintain certain balance conditions and assure good performance.

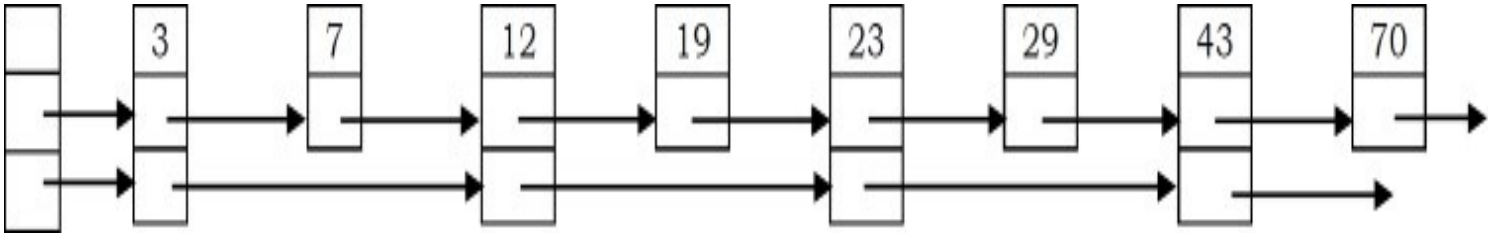
Skip lists are a probabilistic alternative to balanced trees. Skip list is a data structure that can be used as an alternative to balanced binary trees (refer to *Trees* chapter). As compared to a binary tree, skip lists allow quick search, insertion and deletion of elements. This is achieved by using probabilistic balancing rather than strictly enforce balancing. It is basically a linked list with additional pointers such that intermediate nodes can be skipped. It uses a random number generator to make some decisions.

In an ordinary sorted linked list, search, insert, and delete are in O(*n*) because the list must be scanned node-by-node from the head to find the relevant node. If somehow we could scan down the list in bigger steps (skip down, as it were), we would reduce the cost of scanning. This is the fundamental idea behind Skip Lists.

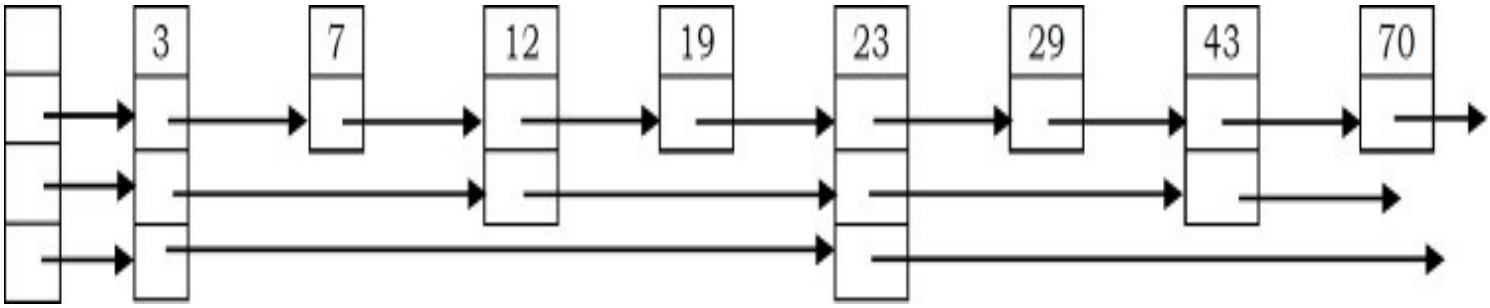
Skip Lists with One Level



Skip Lists with Two Levels



Skip Lists with Three Levels



Performance

In a simple linked list that consists of n elements, to perform a search n comparisons are required in the worst case. If a second pointer pointing two nodes ahead is added to every node, the number of comparisons goes down to $n/2 + 1$ in the worst case.

Adding one more pointer to every fourth node and making them point to the fourth node ahead reduces the number of comparisons to $\lceil n/2 \rceil + 2$. If this strategy is continued so that every node with i pointers points to $2 * i - 1$ nodes ahead, O(*logn*) performance is obtained and the number of pointers has only doubled ($n + n/2 + n/4 + n/8 + n/16 + ... = 2n$).

The find, insert, and remove operations on ordinary binary search trees are efficient, O(*logn*), when the input data is random; but less efficient, O(*n*), when the input data is ordered. Skip List performance for these same operations and for any data set is about as good as that of randomly-built binary search trees - namely O(*logn*).

Comparing Skip Lists and Unrolled Linked Lists

In simple terms, Skip Lists are sorted linked lists with two differences:

- The nodes in an ordinary list have one next reference. The nodes in a Skip List have many *next* references (also called *forward* references).
- The number of *forward* references for a given node is determined probabilistically.

We speak of a Skip List node having levels, one level per forward reference. The number of levels in a node is called the *size* of the node. In an ordinary sorted list, insert, remove, and find operations require sequential traversal of the list. This results in O(*n*) performance per operation. Skip Lists allow intermediate nodes in the list to be skipped during a traversal - resulting in an expected performance of O(*logn*) per operation.

Implementation

```
#include <stdio.h>
#include <stdlib.h>
#define MAXSKIPLEVEL 5
struct ListNode {
   int data;
   struct ListNode *next[1];
};
struct SkipList {
   struct ListNode *header;
   int listLevel;              //current level of list */
};
struct SkipList list;
struct ListNode *insertElement(int data) {
   int i, newLevel;
   struct ListNode *update[MAXSKIPLEVEL+1];
   struct ListNode *temp;
   temp = list.header;
   for (i = list.listLevel; i >= 0; i--) {
      while (temp→next[i] !=list.header && temp→next[i]→data < data)
         temp = temp→next[i];
      update[i] = temp;
   }
   temp = temp→next[0];
   if (temp != list.header && temp→data == data)
      return(temp);
   //determine level
   for (newLevel = 0; rand() < RAND_MAX/2 && newLevel < MAXSKIPLEVEL; newLevel++);
   if (newLevel > list.listLevel) {
      for (i = list.listLevel + 1; i <= newLevel; i++)
         update[i] = list.header;
      list.listLevel = newLevel;
   }
   // make new node
   if ((temp = malloc(sizeof(Node) +
     newLevel*sizeof(Node *))) == 0) {
      printf ("insufficient memory (insertElement)\n");
      exit(1);
   }
   temp→data = data;
   for (i = 0; i <= newLevel; i++) {    // update next links
      temp→next[i] = update[i]→next[i];
      update[i]→next[i] = temp;
   }
   return(temp);
}
// delete node containing data
void deleteElement(int data) {
   int i;
   struct ListNode *update[MAXSKIPLEVEL+1], *temp;
   temp = list.header;
   for (i = list.listLevel; i >= 0; i--) {
      while (temp→next[i] != list.header && temp→next[i]→data < data)
         temp = temp→next[i];
      update[i] = temp;
   }
   temp = temp→next[0];
   if (temp == list.header || !(temp→data == data) return;
   //adjust next pointers
   for (i = 0; i <= list.listLevel; i++) {
      if (update[i]→next[i] != temp) break;
      update[i]→next[i] = temp→next[i];
   }
```

```
    free (temp);
    //adjust header level
    while ((list.listLevel > 0) && (list.header→next[list.listLevel] == list.header))
        list.listLevel--;
}
// find node containing data
struct ListNode *findElement(int data) {
    int i;
    struct ListNode *temp = list.header;
    for (i = list.listLevel; i >= 0; i--) {
        while (temp→next[i] != list.header
          && temp→next[i]→data < data)
            temp = temp→next[i];
    }
    temp = temp→next[0];
    if (temp != list.header && temp→data == data) return (temp);
    return(0);
}
// initialize skip list
void initList() {
    int i;
    if ((list.header = malloc(sizeof(struct ListNode) + MAXSKIPLEVEL*sizeof(struct ListNode *))) == 0) {
        printf ("Memory Error\n");
        exit(1);
    }
    for (i = 0; i <= MAXSKIPLEVEL; i++)
        list.header→next[i] = list.header;
    list.listLevel = 0;
}
/* command-line: skipList maxnum skipList 2000: process 2000 sequential records */
int main(int argc, char **argv) {
    int i, *a, maxnum = atoi(argv[1]);
    initList();
    if ((a = malloc(maxnum * sizeof(*a))) == 0) {
        fprintf (stderr, "insufficient memory (a)\n");
        exit(1);
    }
    for (i = 0; i < maxnum; i++) a[i] = rand();
        printf ("Random, %d items\n", maxnum);
    for (i = 0; i < maxnum; i++) {
        insertElement(a[i]);
    }
    for (i = maxnum-1; i >= 0; i--) {
        findElement(a[i]);
    }
    for (i = maxnum-1; i >= 0; i--) {
        deleteElement(a[i]);
    }
    return 0;
}
```

3.12 Linked Lists: Problems & Solutions

Problem-1 Implement Stack using Linked List.

Solution: Refer to *Stacks* chapter.

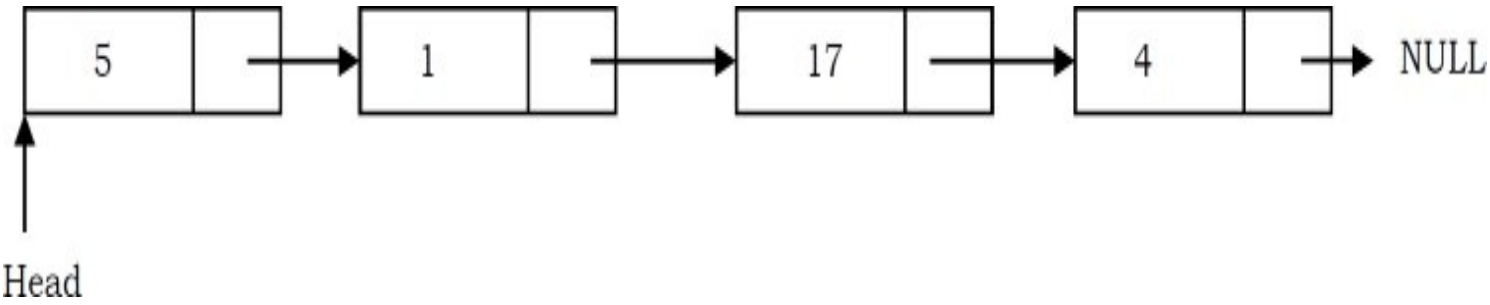
Problem-2 Find n^{th} node from the end of a Linked List.

Solution: Brute-Force Method: Start with the first node and count the number of nodes present after that node. If the number of nodes is $< n - 1$ then return saying "fewer number of nodes in the list". If the number of nodes is $> n - 1$ then go to next node. Continue this until the numbers of nodes after current node are $n - 1$.

Time Complexity: $O(n^2)$, for scanning the remaining list (from current node) for each node.
Space Complexity: O(1).

Problem-3 Can we improve the complexity of Problem-2?

Solution: Yes, using hash table. As an example consider the following list.



In this approach, create a hash table whose entries are < *position of node, node address* >. That means, key is the position of the node in the list and value is the address of that node.

Position in List	Address of Node
1	Address of 5 node
2	Address of 1 node
3	Address of 17 node
4	Address of 4 node

By the time we traverse the complete list (for creating the hash table), we can find the list length. Let us say the list length is M. To find n^{th} from the end of linked list, we can convert this to *M- n* + 1^{th} from the beginning. Since we already know the length of the list, it is just a matter of

returning M- n + 1^{th} key value from the hash table.

Time Complexity: Time for creating the hash table, $T(m) = O(m)$.
Space Complexity: Since we need to create a hash table of size m, $O(m)$.

Problem-4 Can we use the Problem-3 approach for solving Problem-2 without creating the hash table?

Solution: Yes. If we observe the Problem-3 solution, what we are actually doing is finding the size of the linked list. That means we are using the hash table to find the size of the linked list. We can find the length of the linked list just by starting at the head node and traversing the list.

So, we can find the length of the list without creating the hash table. After finding the length, compute $M - n + 1$ and with one more scan we can get the $M - n + 1^{th}$ node from the beginning. This solution needs two scans: one for finding the length of the list and the other for finding $M - n + 1^{th}$ node from the beginning.

Time Complexity: Time for finding the length + Time for finding the $M - n + 1^{th}$ node from the beginning. Therefore, $T(n) = O(n) + O(n) \approx O(n)$. Space Complexity: $O(1)$. Hence, no need to create the hash table.

Problem-5 Can we solve Problem-2 in one scan?

Solution: Yes. Efficient Approach: Use two pointers *pNthNode* and *pTemp*. Initially, both point to head node of the list. *pNthNode* starts moving only after *pTemp* has made n moves.

From there both move forward until *pTemp* reaches the end of the list. As a result *pNthNode* points to n^{th} node from the end of the linked list.

Note: At any point of time both move one node at a time.

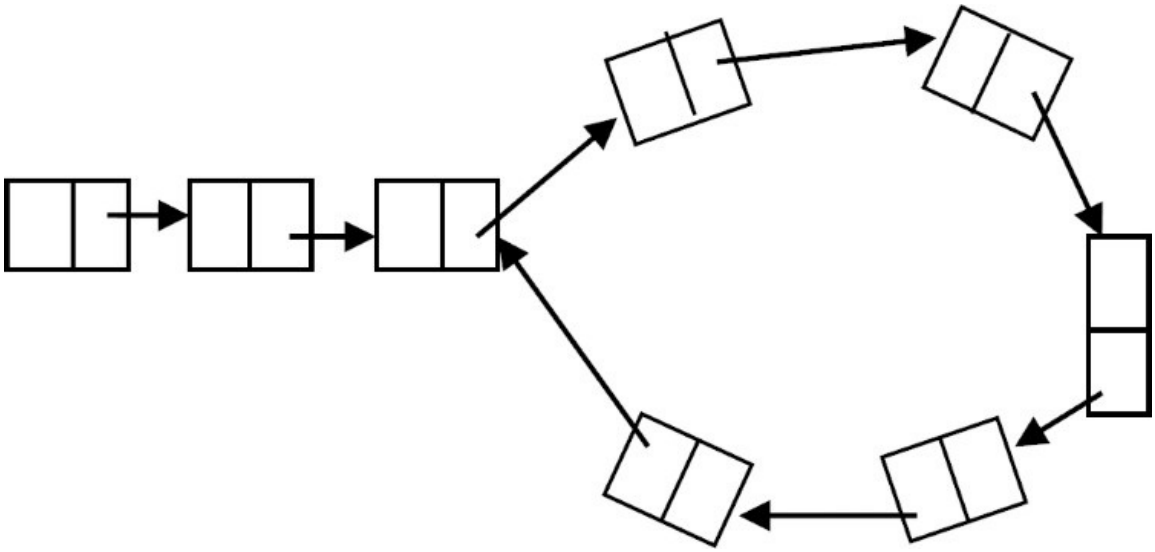
```
struct ListNode *NthNodeFromEnd(struct ListNode *head, int NthNode){
    struct ListNode *pNthNode = NULL, *pTemp = head;
    for(int count =1; count< NthNode;count++) {
        if(pTemp)
            pTemp = pTemp→next;
    }
    while(pTemp) {
        if(pNthNode == NULL)
            pNthNode = head;
        else
            pNthNode = pNthNode→next;
        pTemp = pTemp→next;
    }
    if(pNthNode)
      return pNthNode;
    return NULL;
}
```

Time Complexity: O(*n*). Space Complexity: O(1).

Problem-6 Check whether the given linked list is either NULL-terminated or ends in a cycle (cyclic).

Solution: Brute-Force Approach. As an example, consider the following linked list which has a loop in it. The difference between this list and the regular list is that, in this list, there are two nodes whose next pointers are the same. In regular singly linked lists (without a loop) each node's next pointer is unique.

That means the repetition of next pointers indicates the existence of a loop.



One simple and brute force way of solving this is, start with the first node and see whether there is any node whose next pointer is the current node's address. If there is a node with the same address then that indicates that some other node is pointing to the current node and we can say a loop exists. Continue this process for all the nodes of the linked list.

Does this method work? As per the algorithm, we are checking for the next pointer addresses, but how do we find the end of the linked list (otherwise we will end up in an infinite loop)?

Note: If we start with a node in a loop, this method may work depending on the size of the loop.

Problem-7 Can we use the hashing technique for solving Problem-6?

Solution: Yes. Using Hash Tables we can solve this problem.

Algorithm:

- Traverse the linked list nodes one by one.
- Check if the address of the node is available in the hash table or not.
- If it is already available in the hash table, that indicates that we are visiting the node that was already visited. This is possible only if the given linked list has a loop in it.
- If the address of the node is not available in the hash table, insert that node's address into the hash table.
- Continue this process until we reach the end of the linked list *or* we find the loop.

Time Complexity; O(*n*) for scanning the linked list. Note that we are doing a scan of only the input.
Space Complexity; O(*n*) for hash table.

Problem-8 Can we solve Problem-6 using the sorting technique?

Solution: No. Consider the following algorithm which is based on sorting. Then we see why this

algorithm fails.

Algorithm:

- Traverse the linked list nodes one by one and take all the next pointer values into an array.
- Sort the array that has the next node pointers.
- If there is a loop in the linked list, definitely two next node pointers will be pointing to the same node.
- After sorting if there is a loop in the list, the nodes whose next pointers are the same will end up adjacent in the sorted list.
- If any such pair exists in the sorted list then we say the linked list has a loop in it.

Time Complexity; O(*nlogn*) for sorting the next pointers array.
Space Complexity; O(*n*) for the next pointers array.

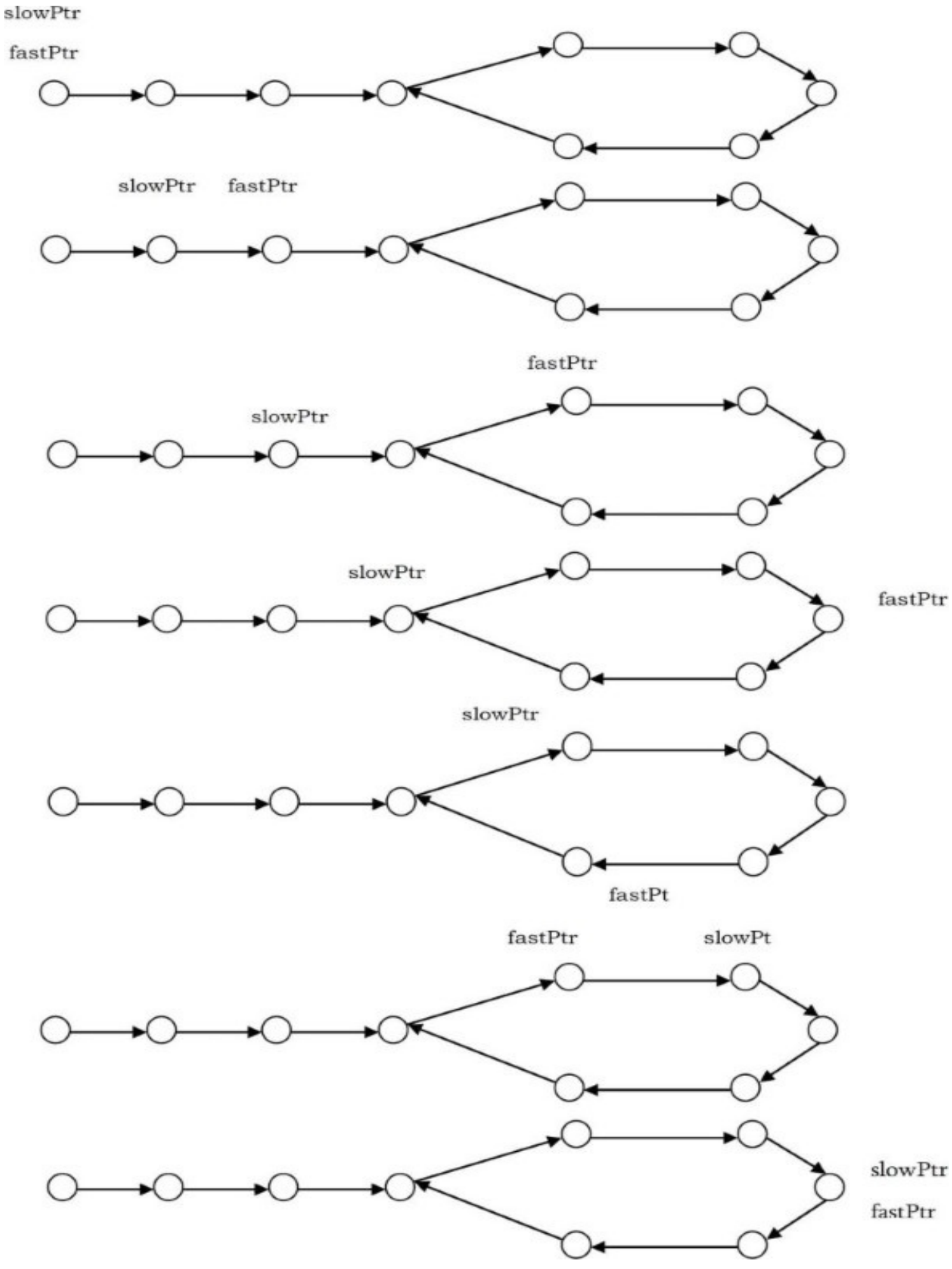
Problem with the above algorithm: The above algorithm works only if we can find the length of the list. But if the list has a loop then we may end up in an infinite loop. Due to this reason the algorithm fails.

Problem-9 Can we solve the Problem-6 in O(*n*)?

Solution: Yes. Efficient Approach (Memoryless Approach): This problem was solved by *Floyd*. The solution is named the Floyd cycle finding algorithm. It uses *two* pointers moving at different speeds to walk the linked list. Once they enter the loop they are expected to meet, which denotes that there is a loop.

This works because the only way a faster moving pointer would point to the same location as a slower moving pointer is if somehow the entire list or a part of it is circular. Think of a tortoise and a hare running on a track. The faster running hare will catch up with the tortoise if they are running in a loop. As an example, consider the following example and trace out the Floyd algorithm. From the diagrams below we can see that after the final step they are meeting at some point in the loop which may not be the starting point of the loop.

Note: *slowPtr* (*tortoise*) moves one pointer at a time and *fastPtr* (*hare*) moves two pointers at a time.


slowPtr
fastPtr
slowPtr
fastPtr
fastPtr
slowPtr
slowPtr
fastPtr
slowPtr
fastPt
fastPtr
slowPt
slowPtr
fastPtr

```
int DoesLinkedListHasLoop(struct ListNode * head) {
  struct ListNode *slowPtr = head, *fastPtr = head;
  while (slowPtr && fastPtr && fastPtr→next) {
    slowPtr = slowPtr→next;
    fastPtr = fastPtr→next→next;
    if (slowPtr == fastPtr)
      return 1;
  }
  return 0;
}
```

Time Complexity: O(*n*). Space Complexity: O(1).

Problem-10 are given a pointer to the first element of a linked list *L*. There are two possibilities for *L:* it either ends (snake) or its last element points back to one of the earlier elements in the list (snail). Give an algorithm that tests whether a given list *L* is a snake or a snail.

Solution: It is the same as Problem-6.

Problem-11 Check whether the given linked list is NULL-terminated or not. If there is a cycle find the start node of the loop.

Solution: The solution is an extension to the solution in Problem-9. After finding the loop in the linked list, we initialize the *slowPtr* to the head of the linked list. From that point onwards both *slowPtr* and *fastPtr* move only one node at a time. The point at which they meet is the start of the loop. Generally we use this method for removing the loops.

```
int FindBeginofLoop(struct ListNode * head) {
        struct ListNode *slowPtr = head, *fastPtr = head;
        int loopExists = 0;
        while (slowPtr && fastPtr && fastPtr→next) {
           slowPtr = slowPtr→next;
           fastPtr = fastPtr→next→next;
           if (slowPtr == fastPtr){
              loopExists = 1;
              break;
           }
        }
        if(loopExists) {
           slowPtr = head;
           while(slowPtr  != fastPtr) {
              fastPtr = fastPtr→next;
              slowPtr = slowPtr→next;
           }
           return slowPtr;
        }
        return NULL;
}
```

Time Complexity: O(n). Space Complexity: O(1).

Problem-12 From the previous discussion and problems we understand that the meeting of tortoise and hare concludes the existence of the loop, but how does moving the tortoise to the beginning of the linked list while keeping the hare at the meeting place, followed by moving both one step at a time, make them meet at the starting point of the cycle?

Solution: This problem is at the heart of number theory. In the Floyd cycle finding algorithm, notice that the tortoise and the hare will meet when they are $n \times L$, where L is the loop length. Furthermore, the tortoise is at the midpoint between the hare and the beginning of the sequence because of the way they move. Therefore the tortoise is $n \times L$ away from the beginning of the sequence as well. If we move both one step at a time, from the position of the tortoise and from the start of the sequence, we know that they will meet as soon as both are in the loop, since they are $n \times L$, a multiple of the loop length, apart. One of them is already in the loop, so we just move the other one in single step until it enters the loop, keeping the other $n \times L$ away from it at all times.

Problem-13 In the Floyd cycle finding algorithm, does it work if we use steps 2 and 3 instead of 1 and 2?

Solution: Yes, but the complexity might be high. Trace out an example.

Problem-14 Check whether the given linked list is NULL-terminated. If there is a cycle, find the length of the loop.

Solution: This solution is also an extension of the basic cycle detection problem. After finding the loop in the linked list, keep the *slowPtr* as it is. The *fastPtr* keeps on moving until it again comes back to *slowPtr*. While moving *fastPtr*, use a counter variable which increments at the rate of 1.

```
int FindLoopLength(struct ListNode * head) {
        struct ListNode *slowPtr = head, *fastPtr = head;
        int loopExists = 0, counter = 0;
        while (slowPtr && fastPtr && fastPtr→next) {
            slowPtr = slowPtr→next;
            fastPtr = fastPtr→next→next;
            if (slowPtr == fastPtr){
                loopExists = 1;
                break;
            }
        }
        if(loopExists) {
            fastPtr = fastPtr→next;
            while(slowPtr  != fastPtr) {
                fastPtr = fastPtr→next;
                counter++;
            }
            return counter;
        }
        return 0;                                   //If no loops exists
}
```

Time Complexity: O(*n*). Space Complexity: O(1).

Problem-15 Insert a node in a sorted linked list.

Solution: Traverse the list and find a position for the element and insert it.

```
struct ListNode *InsertInSortedList(struct ListNode * head, struct ListNode * newNode) {
        struct ListNode *current = head, temp;
        if(!head)
              return newNode;
        // traverse the list until you find item bigger the new node value
        while (current != NULL && current→data <  newNode→data){
              temp = current;
             current = current→next;
       }
        //insert the new node before the big item
        newNode→next = current;
        temp→next = newNode;
        return head;
}
```

Time Complexity: O(*n*). Space Complexity: O(1).

Problem-16 Reverse a singly linked list.

Solution:

```
// Iterative version
struct ListNode *ReverseList(struct ListNode *head) {
  struct ListNode *temp = NULL, *nextNode = NULL;
  while (head) {
    nextNode = head→next;
    head→next = temp;
    temp = head;
    head = nextNode;
  }
  return temp;
}
```

Time Complexity: O(*n*). Space Complexity: O(1).

Recursive version: We will find it easier to start from the bottom up, by asking and answering tiny questions (this is the approach in The Little Lisper):

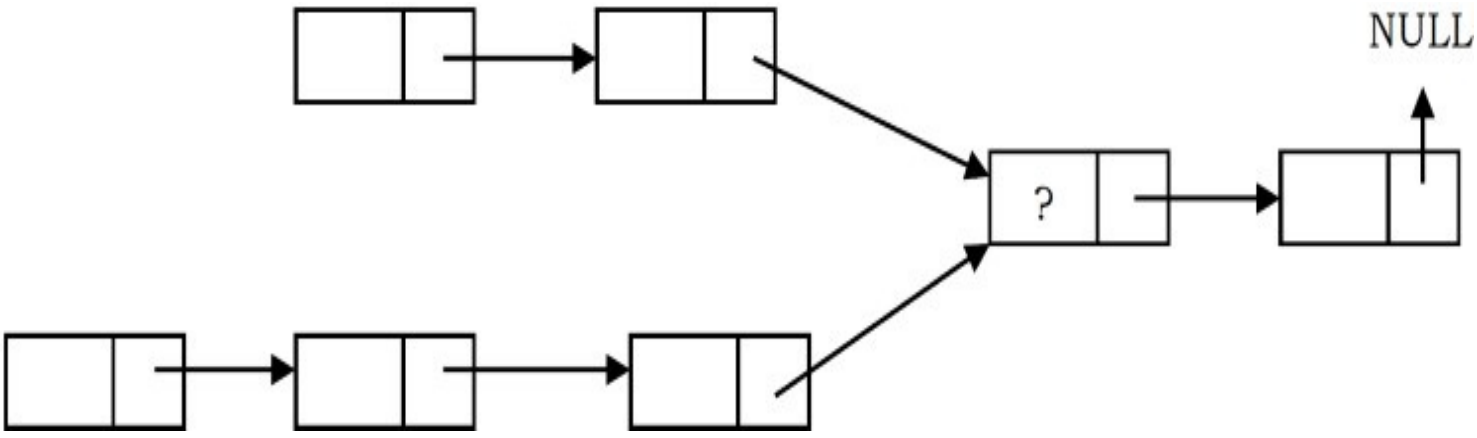
- What is the reverse of NULL (the empty list)? NULL.
- What is the reverse of a one element list? The element itself.

- What is the reverse of an *n* element list? The reverse of the second element followed by the first element.

```
struct ListNode * RecursiveReverse(struct ListNode *head) {
  if (head == NULL)
        return NULL;
  if (head→next == NULL)
        return list;
  struct ListNode *secondElem = head→next;
  // Need to unlink list from the rest or you will get a cycle
  head→next = NULL;
  // reverse everything from the second element on
  struct ListNode *reverseRest = RecursiveReverse(secondElem);
  secondElem→next = head;        // then we join the two lists
  return reverseRest;
}
```

Time Complexity: O(*n*). Space Complexity: O(*n*),for recursive stack.

Problem-17 Suppose there are two singly linked lists both of which intersect at some point and become a single linked list. The head or start pointers of both the lists are known, but the intersecting node is not known. Also, the number of nodes in each of the lists before they intersect is unknown and may be different in each list. *List*1 may have n nodes before it reaches the intersection point, and *List*2 might have *m* nodes before it reaches the intersection point where *m* and n may be $m = n, m < n$ or $m > n$. Give an algorithm for finding the merging point.



Solution: Brute-Force Approach: One easy solution is to compare every node pointer in the first list with every other node pointer in the second list by which the matching node pointers will lead us to the intersecting node. But, the time complexity in this case will be O(*mn*) which will be high.

Time Complexity: O(*mn*). Space Complexity: O(1).

Problem-18 Can we solve Problem-17 using the sorting technique?

Solution: No. Consider the following algorithm which is based on sorting and see why this algorithm fails.

Algorithm:

- Take first list node pointers and keep them in some array and sort them.
- Take second list node pointers and keep them in some array and sort them.
- After sorting, use two indexes: one for the first sorted array and the other for the second sorted array.
- Start comparing values at the indexes and increment the index according to whichever has the lower value (increment only if the values are not equal).
- At any point, if we are able to find two indexes whose values are the same, then that indicates that those two nodes are pointing to the same node and we return that node.

Time Complexity: Time for sorting lists + Time for scanning (for comparing)
= O(*mlogm*) +O(*nlogn*) +O(*m* + *n*) We need to consider the one that gives the maximum value.

Space Complexity: O(1).

Any problem with the above algorithm? Yes. In the algorithm, we are storing all the node pointers of both the lists and sorting. But we are forgetting the fact that there can be many repeated elements. This is because after the merging point, all node pointers are the same for both the lists. The algorithm works fine only in one case and it is when both lists have the ending node at their merge point.

Problem-19 Can we solve Problem-17 using hash tables?

Solution: Yes.

Algorithm:

- Select a list which has less number of nodes (If we do not know the lengths beforehand then select one list randomly).
- Now, traverse the other list and for each node pointer of this list check whether the same node pointer exists in the hash table.
- If there is a merge point for the given lists then we will definitely encounter the node pointer in the hash table.

Time Complexity: Time for creating the hash table + Time for scanning the second list = O(*m*) + O(*n*) (or O(*n*) + O(*m*), depending on which list we select for creating the hash table. But in both

cases the time complexity is the same. Space Complexity: O(n) or O(m).

Problem-20 Can we use stacks for solving the Problem-17?

Solution: Yes.

Algorithm:

- Create two stacks: one for the first list and one for the second list.
- Traverse the first list and push all the node addresses onto the first stack.
- Traverse the second list and push all the node addresses onto the second stack.
- Now both stacks contain the node address of the corresponding lists.
- Now compare the top node address of both stacks.
- If they are the same, take the top elements from both the stacks and keep them in some temporary variable (since both node addresses are node, it is enough if we use one temporary variable).
- Continue this process until the top node addresses of the stacks are not the same.
- This point is the one where the lists merge into a single list.
- Return the value of the temporary variable.

Time Complexity: O($m + n$), for scanning both the lists.
Space Complexity: O($m + n$), for creating two stacks for both the lists.

Problem-21 Is there any other way of solving Problem-17?

Solution: Yes. Using "finding the first repeating number" approach in an array (for algorithm refer to *Searching* chapter).

Algorithm:

- Create an array A and keep all the next pointers of both the lists in the array.
- In the array find the first repeating element [Refer to *Searching* chapter for algorithm].
- The first repeating number indicates the merging point of both the lists.

Time Complexity: O($m + n$). Space Complexity: O($m + n$).

Problem-22 Can we still think of finding an alternative solution for Problem-17?

Solution: Yes. By combining sorting and search techniques we can reduce the complexity.

Algorithm:

- Create an array A and keep all the next pointers of the first list in the array.
- Sort these array elements.
- Then, for each of the second list elements, search in the sorted array (let us assume

that we are using binary search which gives O(*logn*)).

- Since we are scanning the second list one by one, the first repeating element that appears in the array is nothing but the merging point.

Time Complexity: Time for sorting + Time for searching = O(*Max*(*mlogm, nlogn*)).
Space Complexity: O(*Max*(*m, n*)).

Problem-23 Can we improve the complexity for Problem-17?

Solution: Yes.

Efficient Approach:

- Find lengths (L1 and L2) of both lists - O(*n*) + O(*m*) = O(*max*(*m, n*)).
- Take the difference *d* of the lengths -- O(1).
- Make *d* steps in longer list -- O(*d*).
- Step in both lists in parallel until links to next node match -- O(*min*(*m, n*)).
- Total time complexity = O(*max*(*m, n*)).
- Space Complexity = O(1).

```
struct ListNode* FindIntersectingNode(struct ListNode* list1, struct ListNode* list2) {
    int L1=0, L2=0, diff=0;
    struct ListNode *head1 = list1, *head2 = list2;
    while(head1!= NULL) {
      L1++;
      head1 = head1→next;
    }
    while(head2!= NULL) {
      L2++;
      head2 = head2→next;
    }
    if(L1 < L2) {
      head1 = list2;  head2 = list1;  diff = L2 - L1;
    }else{
      head1 = list1;  head2 = list2;  diff = L1 - L2;
    }
    for(int i = 0; i < diff; i++)
      head1 = head1→next;
    while(head1 != NULL && head2 != NULL) {
      if(head1 == head2)
        return head1→data;
      head1= head1→next;
      head2= head2→next;
    }
    return NULL;
}
```

Problem-24 How will you find the middle of the linked list?

Solution: Brute-Force Approach: For each of the node, count how many nodes are there in the list, and see whether it is the middle node of the list.

Time Complexity: $O(n^2)$. Space Complexity: O(1).

Problem-25 Can we improve the complexity of Problem-24?

Solution: Yes.

Algorithm:

- Traverse the list and find the length of the list.
- After finding the length, again scan the list and locate $n/2$ node from the beginning.

Time Complexity: Time for finding the length of the list + Time for locating middle node = $O(n) + O(n) \approx O(n)$.
Space Complexity: $O(1)$.

Problem-26 Can we use the hash table for solving Problem-24?

Solution: Yes. The reasoning is the same as that of Problem-3.

Time Complexity: Time for creating the hash table. Therefore, $T(n) = O(n)$.
Space Complexity: $O(n)$. Since we need to create a hash table of size n.

Problem-27 Can we solve Problem-24 just in one scan?

Solution: Efficient Approach: Use two pointers. Move one pointer at twice the speed of the second. When the first pointer reaches the end of the list, the second pointer will be pointing to the middle node.

Note: If the list has an even number of nodes, the middle node will be of $\lfloor n/2 \rfloor$.

```
struct ListNode * FindMiddle(struct ListNode *head) {
        struct ListNode *ptr1x, *ptr2x;
        ptr1x = ptr2x = head;
        int i=0;
        // keep looping until we reach the tail (next will be NULL for the last node)
        while(ptr1x→next != NULL) {
           if(i == 0) {
              ptr1x = ptr1x→next; //increment only the 1st pointer
              i=1;
           }
           else if( i == 1) {
              ptr1x = ptr1x→next; //increment both pointers
              ptr2x = ptr2x→next;
              i = 0;
           }
        }
        return ptr2x;       //now return the ptr2 which points to the middle node
}
```

Time Complexity: O(n). Space Complexity: O(1).

Problem-28 How will you display a Linked List from the end?

Solution: Traverse recursively till the end of the linked list. While coming back, start printing the elements.

```
//This Function will print the linked list from end
void PrintListFromEnd(struct ListNode *head) {
   if(!head)
      return;

   PrintListFromEnd(head→next);
   printf("%d ",head→data);
}
```

Time Complexity: O(n). Space Complexity: O(n) → for Stack.

Problem-29 Check whether the given Linked List length is even or odd?

Solution: Use a 2x pointer. Take a pointer that moves at 2x [two nodes at a time]. At the end, if the length is even, then the pointer will be NULL; otherwise it will point to the last node.

```
int IsLinkedListLengthEven(struct ListNode * listHead) {
   while(listHead && listHead→next)
      listHead = listHead→next→next;
   if(!listHead)
      return 0;
   return 1;
}
```

Time Complexity: O(⌊n/2⌋) ≈ O(n). Space Complexity: O(1).

Problem-30 If the head of a Linked List is pointing to *kth* element, then how will you get the elements before *kth* element?

Solution: Use Memory Efficient Linked Lists [XOR Linked Lists].

Problem-31 Given two sorted Linked Lists, how to merge them into the third list in sorted order?

Solution: Assume the sizes of lists are m and n.

Recursive:

```
struct ListNode *MergeSortedList(struct ListNode *a, struct ListNode *b) {
   struct ListNode *result = NULL;
   if(a == NULL) return b;
   if(b == NULL) return a;
   if(a→data <= b→data) {
      result =a;
      result→next = MergeSortedList(a→next, b);
   }
   else {
      result =b;
      result→next = MergeSortedList(b→next,a);
   }
   return result;
}
```

Time Complexity: O(n + m), where n and m are lengths of two lists.

Iterative:

```
struct ListNode *MergeSortedListIterative(struct ListNode *head1, struct ListNode *head2){
    struct ListNode * newNode = (struct ListNode*) (malloc(sizeof(struct ListNode)));
    struct ListNode *temp;
    newNode = new Node;
    newNode→next = NULL;
    temp = newNode;
    while (head1!=NULL and head2!=NULL){
        if (head1→data<=head2→data){
            temp→next = head1;
            temp = temp→next;
            head1 = head1→next;
        }else{
            temp→next = head2;
            temp = temp→next;
            head2 = head2→next;
        }
    }
    if (head1!=NULL)
        temp→next = head1;
    else
        temp→next = head2;

    temp = newNode→next;
    free(newNode);
    return temp;
}
```

Time Complexity: O($n + m$), where n and m are lengths of two lists.

Problem-32 Reverse the linked list in pairs. If you have a linked list that holds 1 → 2 → 3 → 4 → *X*, then after the function has been called the linked list would hold 2 → 1 → 4 → 3 → *X*.

Solution:

Recursive:

```
struct ListNode *ReversePairRecursive(struct ListNode *head) {
        struct ListNode *temp;
        if(head ==NULL || head→next ==NULL)
                return;   //base case for empty or 1 element list
        else {
              //Reverse first pair
                temp = head→next;
                head→next = temp→next;
                temp→next = head;
                head = temp;
                //Call the method recursively for the rest of the list
                head→next→next = ReversePairRecursive(head→next→next);
                return head;
        }
}
```

Iterative:

```
struct ListNode *ReversePairIterative(struct ListNode *head) {
        struct ListNode *temp1=NULL, *temp2=NULL, *current = head;
        while(current != NULL && current→next != NULL)  {
              if (temp1 != null) {
                temp1→next→next = current→next;
              }
              temp1 = current→next;
              current→next = current→next→next;
              temp1.next = current;
              if (temp2 == null)
                 temp2 = temp1;
             current = current→next;
        }
       return temp2;
}
```

Time Complexity: O(*n*). Space Complexity: O(1).

Problem-33 Given a binary tree convert it to doubly linked list.

Solution: Refer *Trees* chapter.

Problem-34 How do we sort the Linked Lists?

Solution: Refer *Sorting* chapter.

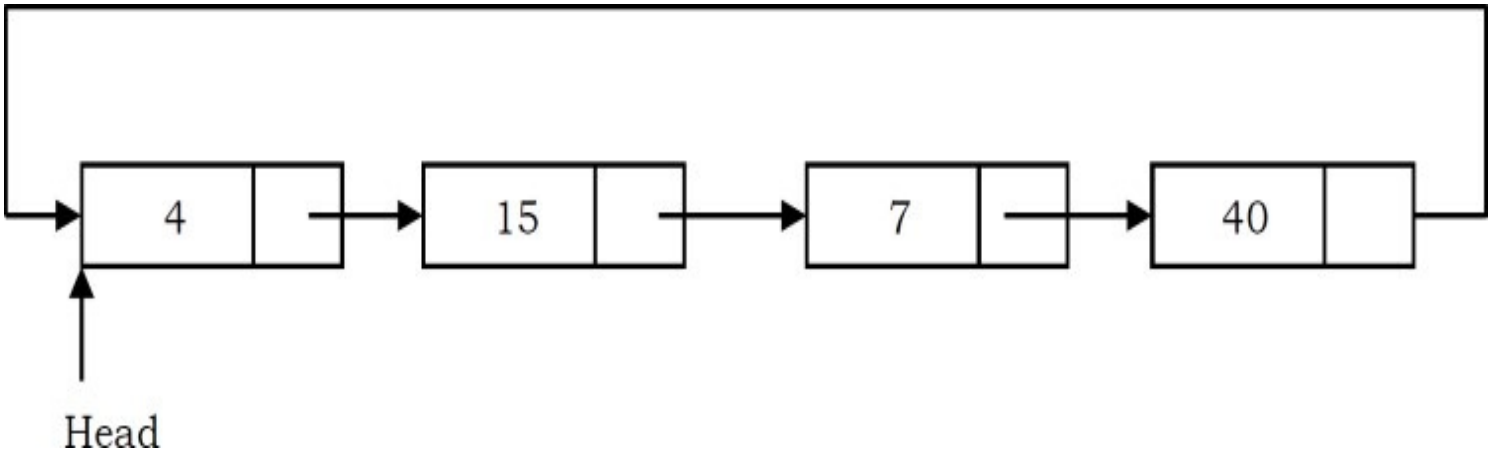
Problem-35 Split a Circular Linked List into two equal parts. If the number of nodes in the list are odd then make first list one node extra than second list.

Solution:

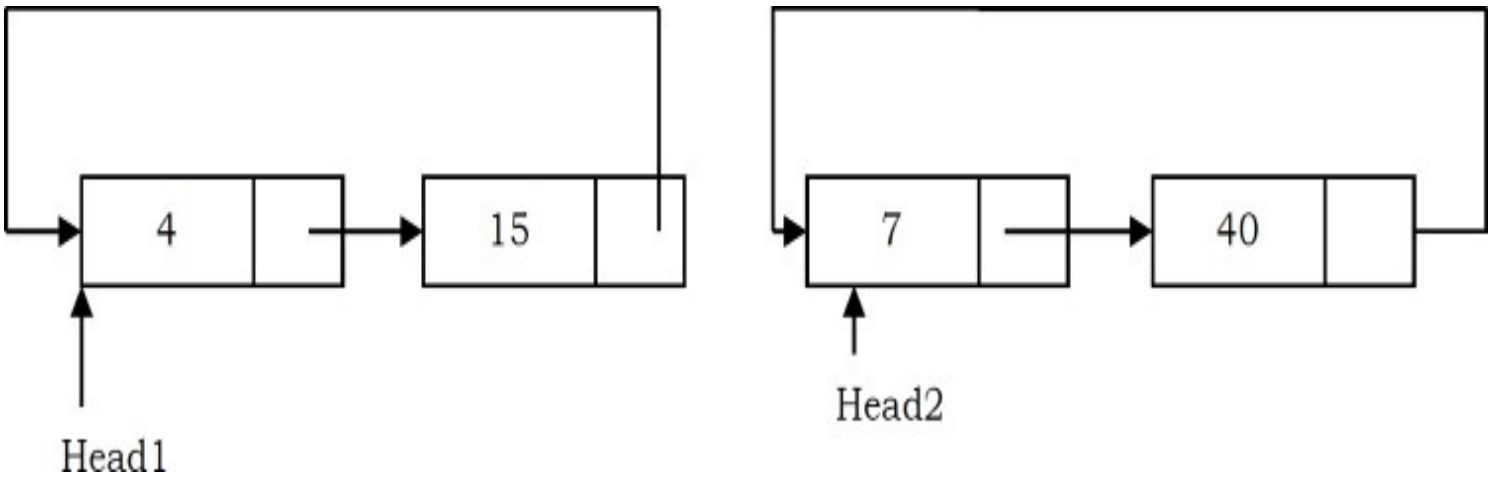
Algorithm:

- Store the mid and last pointers of the circular linked list using Floyd cycle finding algorithm.
- Make the second half circular.
- Make the first half circular.
- Set head pointers of the two linked lists.

As an example, consider the following circular list.



After the split, the above list will look like:



```
// structure for a node
struct ListNode {
        int data;
        struct ListNode *next;
};
void SplitList(struct ListNode *head, struct ListNode **head1, struct ListNode **head2) {
        struct ListNode *slowPtr = head;
        struct ListNode *fastPtr = head;
        if(head == NULL)
                return;
        /* If there are odd nodes in the circular list then fastPtr→next becomes
           head and for even nodes fastPtr→next→next becomes head */
        while(fastPtr→next != head && fastPtr→next→next != head) {
                fastPtr = fastPtr→next→next;
                slowPtr = slowPtr→next;
        }
        // If there are even elements in list then move fastPtr
        if(fastPtr→next→next == head)
                fastPtr = fastPtr→next;
        // Set the head pointer of first half
        *head1 = head;
        // Set the head pointer of second half
        if(head→next != head)
                *head2 = slowPtr→next;
        // Make second half circular
        fastPtr→next = slowPtr→next;
        // Make first half circular
        slowPtr→next = head;
}
```

Time Complexity: O(*n*). Space Complexity: O(1).

Problem-36 If we want to concatenate two linked lists which of the following gives O(1) complexity?

1) Singly linked lists
2) Doubly linked lists
3) Circular doubly linked lists

Solution: Circular Doubly Linked Lists. This is because for singly and doubly linked lists, we

need to traverse the first list till the end and append the second list. But in the case of circular doubly linked lists we don't have to traverse the lists.

Problem-37 How will you check if the linked list is palindrome or not?

Solution:

Algorithm:

1. Get the middle of the linked list.
2. Reverse the second half of the linked list.
3. Compare the first half and second half.
4. Construct the original linked list by reversing the second half again and attaching it back to the first half.

Time Complexity: O(n). Space Complexity: O(1).

Problem-38 For a given K value ($K > 0$) reverse blocks of K nodes in a list.

Example: Input: 1 2 3 4 5 6 7 8 9 10. Output for different K values:
For $K = 2$: 2 1 4 3 6 5 8 7 10 9
For $K = 3$: 3 2 1 6 5 4 9 8 7 10
For $K = 4$: 4 3 2 1 8 7 6 5 9 10

Solution:

Algorithm: This is an extension of swapping nodes in a linked list.

1) Check if remaining list has K nodes.
    a. If yes get the pointer of $K + 1^{th}$ node.
    b. Else return.
2) Reverse first K nodes.
3) Set next of last node (after reversal) to $K + 1^{th}$ node.
4) Move to $K + 1^{th}$ node.
5) Go to step 1.
6) $K - 1^{th}$ node of first K nodes becomes the new head if available. Otherwise, we can return the head.

```
struct ListNode * GetKPlusOneThNode(int K, struct ListNode *head) {
        struct ListNode *Kth;
        int i = 0;
        if(!head)
            return head;
        for (i = 0, Kth = head; Kth && (i < K); i++, Kth = Kth→next);
        if(i == K && Kth != NULL)
                return Kth;
        return head→next;
}
int HasKnodes(struct ListNode *head, int K) {
        int i =0;
        for(i = 0; head && (i < K); i++, head = head→next);
        if(i == K)
            return 1;
        return 0;
}
struct ListNode *ReverseBlockOfK-nodesInLinkedList(struct ListNode *head, int K) {
        struct ListNode *cur = head, *temp, *next, newHead;
        int i;
        if(K==0 || K==1)
            return head;
        if(HasKnodes(cur, K-1))
                newHead = GetKPlusOneThNode(K-1, cur);
        else
                newHead = head;
        while(cur && HasKnodes(cur, K)) {
                temp = GetKPlusOneThNode(K, cur);
                i=0;
                while(i < K) {
                    next = cur→next;
                    cur→next=temp;
                    temp = cur;
                    cur = next;
                    i++;
                }
        }
        return newHead;
}
```

Problem-39 Is it possible to get O(1) access time for Linked Lists?

Solution: Yes. Create a linked list and at the same time keep it in a hash table. For n elements we have to keep all the elements in a hash table which gives a preprocessing time of O(n).To read any element we require only constant time O(1) and to read n elements we require n * 1 unit of time = n units. Hence by using amortized analysis we can say that element access can be performed within O(1) time.

Time Complexity – O(1) [Amortized]. Space Complexity - O(n) for Hash Table.

Problem-40 **Josephus Circle:** N people have decided to elect a leader by arranging themselves in a circle and eliminating every M^{th} person around the circle, closing ranks as each person drops out. Find which person will be the last one remaining (with rank 1).

Solution: Assume the input is a circular linked list with N nodes and each node has a number (range 1 to N) associated with it. The head node has number 1 as data.

```
struct ListNode  *GetJosephusPosition(){
        struct ListNode *p, *q;
        printf("Enter N (number of players): ");
        scanf("%d", &N);
        printf("Enter M (every M-th payer gets eliminated): ");
        scanf("%d", &M);
        // Create circular linked list containing all the players:
        p = q = malloc(sizeof(struct node));
        p→data = 1;
        for (int i = 2; i <= N; ++i) {
                p→next = malloc(sizeof(struct node));
                p = p→next;
                p→data = i;
        }
       // Close the circular linked list by having the last node point to the first.
        p→next = q;
        // Eliminate every M-th player as long as more than one player remains:
        for (int count = N; count > 1; --count) {
                for (int i = 0; i < M - 1; i++)
                        p = p→next;
                p→next = p→next→next;  // Remove the eliminated player from the circular linked list.
        }
        printf("Last player left standing (Josephus Position) is %d\n.", p→data);
}
```

Problem-41 Given a linked list consists of data, a next pointer and also a random pointer which points to a random node of the list. Give an algorithm for cloning the list.

Solution: We can use a hash table to associate newly created nodes with the instances of node in the given list.

Algorithm:

- Scan the original list and for each node *X*, create a new node *Y* with data of *X*, then store the pair (*X*, *Y*) in hash table using *X* as a key. Note that during this scan set *Y* → *next* and *Y* → *random* to *NULL* and we will fix them in the next scan.
- Now for each node *X* in the original list we have a copy *Y* stored in our hash table. We scan the original list again and set the pointers building the new list.

```
struct ListNode *Clone(struct ListNode *head){
    struct ListNode *X, *Y;
    struct HashTable *HT = CreateHashTable();
    X = head;
    while (X != NULL) {
        Y = (struct ListNode *)malloc(sizeof(struct ListNode *));
        Y→data = X→data;
        Y→next = NULL;
        Y→random = NULL;
        HT.insert(X, Y);
        X = X→next;
    }
    X = head;
    while (X != NULL) {
        // get the node Y corresponding to X from the hash table
        Y = HT.get(X);
        Y→next = HT.get(X→next);
        Y.setRandom = HT.get(X→random);
        X = X→next;
    }
    // Return the head of the new list, that is the Node Y
    return HT.get(head);
}
```

Time Complexity: O(*n*). Space Complexity: O(*n*).

Problem-42 Can we solve Problem-41 without any extra space?

Solution: Yes.

```
void Clone(struct ListNode *head){
    struct ListNode *temp, *temp2;
    //Step1: put temp→random in temp2→next,
    //so that we can reuse the temp→random field to point to temp2.
    temp = head;
    while (temp != NULL) {
      temp2 = (struct ListNode *)malloc(sizeof(struct ListNode *));
      temp2→data = temp→data;
      temp2→next = temp→random;
      temp→random = temp2;
      temp = temp→next;
    }
    //Step2: Setting temp2→random.  temp2→next is the old copy of the node that
    // temp2→random should point to, so temp→next→random is the new copy.
    temp = head;
    while (temp != NULL) {
      temp2 = temp→random;
      temp2→random = temp→next→random;
      temp = temp→next;
    }
    //Step3: Repair damage to old list and fill in next pointer in new list.
    temp = head;
    while (temp != NULL) {
    temp2 = temp→random;
    temp→random = temp2→next;
    temp2→next = temp→next→random;
    temp = temp→next;
    }
}
```

Time Complexity: O(3n) ≈ O(*n*). Space Complexity: O(1).

Problem-43 We are given a pointer to a node (*n*ot the tail node) in a singly linked list. Delete that node from the linked list.

Solution: To delete a node, we have to adjust the next pointer of the previous node to point to the